

● INSIDOR

Design Document

The product, the three settled decisions, and every surface — how it works, how it looks, its states and its build order.

DESIGN.MD · 27 JULY 2026
30,409 WORDS · GENERATED FROM THE REPOSITORY, NOT MAINTAINED SEPARATELY

Contents

The one rule

Who it is for

What makes it hard

The three product decisions

How the system works

The stack

The structure

Navigation

01. Live Feed — the posts lane

02. Live Feed — the coins lane, and Trending

03. The Story page

How it looks

How it works

States

What it needs

Build order

04. The Coin page — chart, swap, trade

05. Creating coins

06. Search and Watchlist

07. Wallet, onboarding, and the operator surface

4.1 The primary-action resolver

4.2 The coin-confidence stamp

4.3 The market-data pending state

4.4 The lead-time claim

4.5 The fee line, and the staleness band

Status: for review · 27 July 2026 **Supersedes:** every earlier design and architecture document in this repo and in the `Insid0or` folder. **Built from:** three adversarial research runs (65 agents, ~6M tokens) — stack verification against primary sources, end-to-end system design, and feature specification with hostile review. Raw output in `docs/research/` and `docs/design/specs/`.

How to read this. Section 1 is the product. Section 2 is what has been decided and is no longer up for debate. Section 3 is every screen, how it works and how it looks — the bulk of the document. Sections 4 to 6 are the parts that cut across features, what is deliberately excluded from v1, and the ten decisions still waiting on the founder.

This is a full rebuild. Nothing in the current repository is a foundation. `apps/web` and `apps/pipeline` are written from scratch; `site/` , `worker/` and the root `api/` are deleted once their replacements land. A handful of things carry over as *notes* rather than code — the TikTok meme-scoring prompt, the generic-ticker denylist, the per-tweet billing arithmetic — because they are expensive to rediscover, not because they are worth preserving.

1. The product in one page

Every memecoin starts as a viral moment. The coin is downstream of the story.

Axiom, Photon and DexScreener all start at the coin, so by the time they show you a token you are racing ten thousand people to something that is already priced. **Insidor starts at the story** — roughly 20 to 40 minutes earlier. That head start is the entire product. Trading is table stakes; discovery is the moat.

The one rule

Every surface resolves to a single action, and the primary button is a **pure function of how many confirmed coins a story has**:

CONFIRMED COINS	PRIMARY ACTION
0	Create coin
1	Buy \$TICKER
2 or more	See the N coins

The ambiguous "buy which one?" state is never rendered — not because the copy is careful, but because it is unrepresentable in the component. An **unsure** match is not a confirmed coin: the candidates are shown and clearly labelled unverified, Buy is suppressed, and the primary action stays Create.

Who it is for

The **trader** who wants to be first, not fast. The **creator** who wants to mint off a meme and earn the fees. The **normie** who saw something on TikTok and wants in without learning what a bonding curve is.

What makes it hard

The claim has to be provable. The source post is public and timestamped, so anyone can check it. That is why misses stay on the record — a track record with no left tail reads as a casino ad.

Earliness and correctness are in direct tension, and that is the whole engineering problem. Fire early and you have less evidence. Wait for certainty and you have spent the edge that *is* the product. The 9-minute detection floor, the two-author birth quorum and the abstain band are not three problems — they are the same trade in three places.

The failure modes are asymmetric. A story you miss costs a user nothing; they never knew. A wrong Buy button costs them money. Precision beats recall structurally — but commercial pressure runs the other way, which is why the precision floor is written down rather than left to judgement.

Not everything viral is coinable, and that boundary is taste rather than logic.

The story is the spine. One story spawns many coins. Attach discussion, watchlists and the record to the story and it stays whole while coins come and go beneath it.

2. What is settled

The three product decisions

D1 — Precision over volume. Coin matching targets 0.97 precision with an explicit abstain band of 15–30%. When the system is not confident, the Buy button is suppressed — but the candidate mints are still **shown**, labelled unverified, so the user decides with the uncertainty visible. Abstain never means hide. The cost is real and accepted: roughly one story in four that genuinely has a coin will show Create instead of Buy.

D2 — Misses are visible by default. The record’s default filter is All, never Hits. Negative lead time renders plainly, in the same weight as positive, and is never colour-flagged red — colouring it red creates the incentive to hide it. Every rate is shown against an explicit denominator.

D3 — Coinability has three tiers, judged on *is there an identifiable person harmed by this becoming a coin*:

TIER	BEHAVIOUR
Never surface	Identifiable private individuals · deaths · violence against a named person · anything involving minors
Surface, no Create	Public tragedy, ongoing disasters — discussable, not mintable
Normal	Everything else

This lives in the scoring prompt, not a keyword blocklist.

How the system works

Posts are discovered **by engagement floor across every account** — a low floor (~15 likes) over a short window, which discards ~99% of X while staying completely account-agnostic. A curated stream of ~500 accounts sits on top purely as a **latency optimisation**, not as the coverage mechanism, and is an unproven bet pending a shadow run.

Each tracked post is snapshotted on a geometric grid (4, 9, 14, 21, 30, 42, 58, 78 minutes). The trigger is a **per-author-normalised engagement rate with acceleration**, never raw views. Two observations are required before any rate exists, which sets a hard floor of $\tau \approx 9 \text{ minutes}$ and a realistic discovery latency of **p50 ~12 min, p90 ~22 min**. That is physics, not engineering.

Posts cluster into stories by cashtag, then entity and keyword overlap, then embedding similarity, with an LLM adjudicator confined to an ~8–12% uncertainty band. A story becomes display-eligible at **promote**, and `promoted_at` is set once and never moved — it is what lead time is measured from.

Earliness is proven against two prospective clocks: every Solana mint via websocket, and ~300 crypto-Twitter accounts polled every two minutes. `t_crypto = min(t_mint, t_ct)` . Both are written *before* any alert fires, because a retrospective search has lookahead bias and would make the headline number a lie.

Claude appears in exactly one tier of the pipeline — is this a story, is it coinable, what is it called. Everything above is arithmetic on engagement counts; everything below is deterministic gates and a calibrated score. There is no model training, no GPU fleet, and one model we fit ourselves: a logistic regression calibrating match confidence on ~300 hand-labelled pairs.

The stack

Every identifier below was confirmed against a primary source or a live call on 26–27 July 2026. They move weekly; re-verify before writing code against any of them.

LAYER	CHOICE
Frontend	Next.js 16 · TypeScript · Tailwind 4
Data	Supabase Postgres + Realtime Broadcast, sharded per narrative
Wallet	Privy @privy-io/react-auth , Solana subpath
Swap	Jupiter Swap V2 (/swap/v2/order → /execute) — Ultra is deprecated
Charts	DexScreener iframe embed
Creation	Pump.fun create_v2 — mints Token-2022, creator independent of signer
Safety	RugCheck full report — not /report/summary , which carries no authority fields
Ingestion	twitterapi.io at \$0.00015/post · Apify for TikTok

Cost: ~\$120/month at beta, ~\$1,089 at 1k DAU, ~\$4,543 at 50k. Ingestion is the largest line at every stage and is **DAU-independent** — it is a coverage dial the founder sets, not a per-user cost.

The structure

A two-app npm workspace: [apps/web](#) (the only thing Vercel builds) and [apps/pipeline](#) (Node). Inside [apps/web](#) , one folder per feature, each with the same six segments and exactly two public entry points — [index.ts](#) for client-safe, [server.ts](#) for server-only. [app/](#) becomes a routing manifest with zero business logic.

Navigation

Three tabs. **Live feed** — two lanes, stories and coins, with filters and sorts that absorb the mockup's Trending, Narratives, Tokens and New Pairs, which were two tables shown twice with different default sorts. **Search. You.** Story and Coin are pushed routes with their own URLs, not tabs.

Still open: whether the stories lane stays merged into Live feed or becomes its own tab. Merging is cleaner and matches Feature 7's own filter requirement; a separate tab guarantees browse survives even if the filter strip is weak. Written merged, flagged where it matters.

3. The surfaces

A user must be traceable from landing to a filled trade with no dead end:

Live feed → click a story → **Story page** (is this real, am I early?) → the fork → **Coin page** (buy) or **Create** (mint the first) → **You** (what you own, how early you were) → **The record** (did it pay). **Search** is reachable from anywhere and answers the anti-duplicate question: does a coin already exist for this?

01. Live Feed — the posts lane

What it is

A ranked terminal board of the 40 posts going viral right now, ordered by how fast each is accelerating against its own author's baseline — not by how many views it has. One job: *what is blowing up, and how much head start do I still have.* This is the pre-coin surface. Most rows have no ticker and nothing to buy, and that is the point — if you can buy it here, you were late.

How it looks

Three tabs: Live feed · Search · You. Inside Live feed a segmented switch — POSTS · STORIES · COINS — POSTS default. (Whether stories stay a sibling switch here or become their own top-level tab is still open with you; this assumes merged.)

A 44px sticky control strip on near-black glass: lane switch, platform chips, age window (15m · 1h · **4h** · 24h), sort chips (HEAT · RATE · ACCEL · NEW · REACH), threshold popovers reading **MEME** ≥ 0.35 and **ORG** ≥ 0.30 , and one loud red toggle, SHOW RISKY. Right side: COLLAPSE BY STORY, SINCE OPEN, Δ10m, **LAST TICK 4s** , **40 posts** .

Then the board — one dense table on a #0C0C0E card, sticky header, **54px rows**, matching the stories and coins boards so the three lanes read as one instrument. Every row carries a 3px lifecycle rail on its left edge (lime heating, cyan peaking, dim cooling), then eleven columns:

COL	SHOWS
#	Rank 1–40 plus a one-tick caret ▲2 / ▼1 that fades after 20s; in Δ10m mode it is relative to ten minutes ago. Rows never physically slide.
POST	Thumbnail · author · text · chip slot (below).
HEAT	0–100 mono white over a 3px cyan bar. The composite sort key made legible.
REACH	Cumulative views (2.4M) in muted grey, not white — it is an eligibility gate, not the score, and every competitor trains users to read it as the score.
RATE	12.4K/m view arrival, coloured by lifecycle; dim sub-line 48 rt/m , because the reshare rate is what actually gates the post.
ACCEL	Burst as a multiple, 3.2× . Lime ≥1.3, amber 0.8–1.3, dim below. — on a single snapshot, never a fabricated 1.0× . Moves ~40s before the rank does.
20M	22px sparkline of the last 8 view-rate deltas. One vertical step then flat is bought reach, legible in a second.
ORG	Organic multiplier 0–100. Red <30, amber 30–60, muted 60–85, white above.
MEME	meme_score 0–100 plus an amber o when coinability is no_create .
AGE	14m / 2h , with the CT lead on a sub-line.
<i>(action)</i>	118px, one primary button, behind a 1px divider.

The POST cell is four lines. Thumbnail: real image; or a video poster frame with a play overlay and a **from video** chip; or, with no media, a deterministic gradient seeded by post id plus **no media** . **The loremflickr → picsum stock-photo fallback is deleted** — fabricating media on a terminal whose thesis is precision is a lie told in pixels. Author line: platform mark · name · **@handle** · · **412K** followers inline (the plausibility denominator, not a sortable fact). Then the post text, verbatim, one line, CSS-clipped only. TikTok hashtags render dim; when a caption is >65% hashtags we show Insidor's own one-line subject prefixed with a mono **DERIVED** label so it never passes as the author's words.

The chip slot is a fixed 14px line, always present, so chips never reflow the row. At ≤1040px it clamps to two, precedence **RISK → NOT MINTABLE → ALREADY HOT → ADOPTED → FROZEN → 2ND WAVE → → STORY → +4 posts** , overflow **+2** . The honesty labels are never what gets dropped.

Nothing in the post cell is an engagement control — no like, repost, reply, follow. **Create coin** and **Buy** are Insidor actions on an Insidor object, which is why they sit outside that cell behind a divider. Name, handle and timestamp link out in a new tab.

Under the board: **40 of 63 eligible · 12 hidden by gate** , opening **meme_score 7 · 0 3 · author cap 2 · coinability 4** . On wide viewports a 460px rail logs board events: **ENTERED #7 · @handle** , **LEFT · cooling** , **2ND WAVE · #12** , **CT PICKUP · 3 accts** , **MATCH RETRACTED · \$KANG** , **NORMS REBASED** , **SNAPSHOT GAP · 4m** .

How it works

```
base      = rate_LCB_norm × burst^0.5
PostHeat = (base × 0)^0.85 / ((age_min + 12)/12)^1.35 × C × E
```

rate_LCB_norm is a Gamma-Poisson lower confidence bound on the **view arrival rate**, normalised by the platform's 24h median. The prior comes from the author's follower-decile bucket, which *is* the cold-start answer — no null, no blind window. The lower bound punishes small n, which kills the 3→30 false positive and prices in bought views.

burst = S_fast / S_slow , continuous-time EWMA, τ 20min over 6h: **S ← S·exp(-Δt/τ) + (1 - exp(-Δt/τ))·(ΔV/Δt)** . Continuous, not **a·x + (1-a)·S** , because the snapshot grid is irregular (4/9/14/21/30/42/58/78 min) and the discrete form biases hot-tier posts upward by construction. Burst is scale-free — a 300-follower account spiking outranks a 500k account's normal traffic — and it detects a **second wave**, which an HN-shaped score provably cannot and which is the dominant pattern in memecoin attention.

0 ∈ [0.15, 1.0] multiplies five tests: step detection on the delta distribution, likes/views depth, replies/likes conversation, audience plausibility, monotonicity. Failing one caps the score; floored at 0.15 not 0, so a false positive demotes rather than erases. **Plausibility is exempt on TikTok** — the parser hardcodes **quotes: 0** and For-You routinely puts views at

200× followers organically, so leaving it armed applies 0.35 to every TikTok breakout and gates out virality on the platform where memes originate.

`C` = 0.35 once three crypto-native accounts have quoted the post: when CT finds it our edge is gone and it should fall off the board *while still growing*. `E` = 0.70 on censored entry (first seen already large, or >20 min after posting); those rows get an amber `ALREADY HOT` chip, no lead figure, and permanent exclusion from the earliness denominator. `Δ 0.70` **is judgement, not a fitted value** — the backfill settles it.

Displayed heat is `round(100 · min(1, PostHeat / p95_heat_1h))`, self-calibrating so a dead night still shows spread. That denominator rebases hourly, so a row can jump 61 → 88 with nothing changing underneath — the tick animation is **suppressed on the first frame after a rebase**, with a dim `NORMS REBASED` rail event. The HEAT popover decomposes `base · burst · O · C · E · age penalty` **plus the denominator and its updated_at**.

Eligibility, applied to every sort and filter identically: age ≤ 240min, reshare-rate LCB above an absolute floor (reshares, never views), a view floor, meme ≥ 0.35, O ≥ 0.30, coinability ≠ blocked, author <3× in the top 20, two qualifying ticks. Sorting by NEW still gates — otherwise "newest" returns 40,000 pieces of junk and the product looks broken. SHOW RISKY is the one exception, which is why it is loud and never persisted.

Stability. Commit every 20s: a challenger must beat an incumbent by 8% to pass it, two ticks to enter, three to leave, 90s dwell, max 5 positions per tick. Then freeze-on-hover (reorders queue, strip reads `PAUSED · 3 QUEUED`), never reflow below the fold (entrants above row 12 batch into `↑ 3 new`), and the load-bearing rule: **values update live, positions do not**. Rows live in a `Map<post_id, Row>` updated on every push; only the ordering array is held.

The action. The primary button is a pure function of *n* — coins on the post's story that are `verdict = confirmed` and pass the §4.4 hard safety gates (mint authority, freeze authority, top holders >60%, live with <\$3k liquidity, honeypot proxy). Confirmed answers "is this the right coin", not "is this safe to buy". `n=0` → `Create coin`; `n=1` → `Buy $TICKER`; `n≥2` → `See the 3 coins`.

n is **market-data independent**: a coin confirmed four minutes ago with a null mcap is still `n=1`, still Buy, with mcap and liquidity as a dim `-`. `lib/stories.ts`'s `tradeable()` / `storyAction()` pair, which requires a non-null mcap, is deleted — under it a four-minute-old mint flips back to Create and drives a duplicate mint in the exact window this product exists to serve.

An **UNSURE** match makes Buy unrenderable; `Create coin` stays primary and the candidate shows as a muted `$KANG · UNVERIFIED` chip. Abstain never means hide, and confidence is categorical — never "73% match" beside an actionable control, because users treat 73% as good enough and do their own thresholding. **Confirmed but unsafe** → `See the coin` (outline) plus a red `RISK · mint authority` chip; only SHOW RISKY promotes it to Buy. **Adopted** (a pre-existing mint a story attaches to) → Buy plus a mandatory amber `ADOPTED · 604d`; without it a 604-day-old shell with \$3k liquidity renders as an ordinary buy.

Fees are disclosed before every confirm. The action cell carries a dim `+0.5% fee` second line, read from config, never hardcoded — the Jupiter integrator fee has a 50 bps floor, so 0.5% is the cheapest possible cost per trade and we say so rather than implying zero. The trade modal renders expected received, price impact and max slippage before confirm goes live, red above threshold; `dynamicSlippage` is already on and this board specialises in sub-10-minute curves. Create shows the pump.fun creator-fee share and any Insider share before submit.

`Create coin` prefills ticker, name and media from the LLM suggestion. **That is the only place the suggested ticker may exist** — it must never reach enrichment or `narrative_tickers`, which is the live \$KANG bug at `cluster.js:310`.

Lead time. An unclustered post has no narrative, so no `promoted_at`, so no computable lead — we suppress the figure there rather than invent a second clock that silently means something else. Once CT picks it up, AGE's sub-line renders cyan `+31m early`, dim `CT +0m` (a genuine tie only), or red `CT -7m late`. Roughly a third of real detections are late; the number that proves the claim has to render its own failure.

Story collapse. A row never vanishes because a job clustered it. On clustering it stays put, gains `→ STORY`, and the button re-resolves — `Create coin` → `Buy $TICKER` in place with a one-time 850ms cyan pulse, the most important state change here. The reverse is equally visible: retraction reverts to Create, drops the candidate to the unverified chip, and logs an amber `MATCH RETRACTED` event — never silent, never the celebratory pulse. Later members fold into one row as `+4 posts`. With COLLAPSE off, members render individually with a colour-keyed left marker and **share one resolved action state** — twelve rows of one story is not twelve independent Buy buttons.

States

STATE	WHAT THE USER SEES
First run	One-time dismissible strip naming the three columns that decide a row — HEAT, ACCEL, AGE lead. Every hover popover has a tap equivalent.
Loading	12 skeleton rows; header and strip render immediately, already interactive.
Empty — quiet	<code>No posts currently qualify.</code> plus LAST TICK age and the gate breakdown <i>including a bare coinability count</i> , so a filtered window is never confusable with a slow night.
Empty — broken	LAST TICK >120s: amber <code>PIPELINE STALE — last snapshot 6m ago. Ranks are frozen.</code> Last tick still rendered, HEAT dimmed.
Error	Red banner, retry, last good tick from cache stamped <code>as of 14:22</code> .
Cold start (1 snapshot)	ACCEL <code>—</code> , blank sparkline, HEAT with a dim asterisk. Enters after 2 ticks unless the hard rate floor fires the escape hatch (lime <code>NEW</code> , 120s).
Censored entry	Amber <code>ALREADY HOT</code> , E=0.70, AGE sub-line blank. Never a fake <code>+31m early</code> .
Second wave	Cyan <code>2ND WAVE</code> ; original posted_at kept so the age penalty stays honest.
Negative lead	Red <code>CT -7m late</code> on the row, in the sheet subtitle, and as a <code>CT BEAT US</code> rail event.
Story with no coin	The normal case. <code>Create coin</code> , prefilled.
Unverified match	Buy unrenderable . <code>Create coin</code> primary; muted <code>\$KANG · UNVERIFIED</code> , no one-tap path to buying.
Confirmed, no market data	<code>Buy \$TICKER</code> normally; mcap and liquidity dim <code>—</code> , never <code>\$0</code> , never a reassuring green.
Confirmed but unsafe	<code>See the coin</code> outline + red <code>RISK</code> naming the failing gate.
Adopted coin	Buy + amber <code>ADOPTED · 604d</code> ; sheet reads "existing coin, adopted by this story".
Match retracted live	Reverts to Create, candidate drops to unverified, amber rail event.
<code>no_create</code>	Ranks and displays normally; action cell reads <code>NOT MINTABLE</code> + reason. Discussable, not mintable.
<code>blocked</code>	Never enters board or rail. Counted, uncatalogued, in the gate breakdown.
Wallet not connected	Buttons show their true label , fully enabled; click opens Privy then completes the intent. Never greyed for auth.
Insufficient SOL	Modal states the shortfall inclusive of the 0.5% fee, before any signature request.
Create fails	Ticker taken, mint rejected, signature declined: sheet stays open, reason inline, prefill intact.
Ingest gap	Amber <code>FROZEN</code> , HEAT dimmed, ACCEL <code>—</code> . EWMA frozen, not decayed; window marked unmeasurable, not a win.
Post deleted upstream	Keeps rank, gains dim <code>UNAVAILABLE · as of 14:22</code> , counters stop, action suppressed to Create.
Author capped	Third row suppressed; the author's top row gains a dim <code>+2 more from @handle</code> .
Media fetch failed	Gradient placeholder + <code>no media</code> . Never a stock-photo substitute.
Frozen on hover	Cyan inset ring, <code>PAUSED · 3 QUEUED</code> . Values tick, positions hold.
Polling fallback	Live dot amber, rail reads <code>polling fallback</code> , row age from the response Age header.
Scoring paused	Existing budget-exhausted banner verbatim; existing rows keep updating.

What it needs

Used as-is: `narrative_posts`, `post_snapshots`, `post_meme_scores`, `narratives`, `narrative_tickers`, `worker_cycle_log`, `worker_budget_state`.

⚠ Does not exist:

- `post_heat` (PK post_id): heat, base, presented_heat, rate_lcb_views, rate_lcb_reshares, rate_lcb_norm, s_fast, s_slow, ewma_updated_at, burst, organic_o, organic_features, ct_pickup_count, ct_first_seen_at, lifecycle, v_peak, censored_entry, second_wave, frozen, ticks_qualified, computed_at. **Nothing in the formula has a home today.**
- `board_ticks` + `board_state` (rank, prev_rank, entered_at, pinned_until, exit_ticks) — without them there is no rank delta, no enter/exit counting, no Δ10m, no rail log. ⚠ Retention unresolved: ~173k rows/day/lane, and the Δ10m lookback blocks aggressive trimming.
- `platform_norms` (median_view_rate_24h, median_views_at_age, p95_heat_1h) and `author_roster` (follower_decile, a0, b0).

- `coin_matches` (narrative_id, mint, verdict, score, evidence, mint_created_at, matched_at) — the story-to-coin workstream's deliverable, not this one's. `narrative_tickers` has no verdict column and **no `source` column**, so an invented ticker and a real cashtag are indistinguishable downstream.
- `narrative_posts.coinability` enum + reason + model. D3 has no schema representation today.
- `narratives.promoted_at` — the lead-time clock, genuinely absent.
- `followers_at_post` (current `followers` grew *because of* the post — plausibility against it is circular), `media_phash`, `post_derived_subject`, `ingest_gaps`, a real `watchlist` table (today two in-memory Sets), and an index on `post_snapshots(post_id, captured_at desc)`.
- **Not needed:** `media_poster_url` — both parsers already write poster frames into `media_url` with type video/gif. ⚠ Unverified: whether TikTok CDN cover URLs are signed and expiring. Check a 24h-old row.
- Platform chips render from `select distinct platform` — today exactly X and TikTok. Three hardcoded chips reading 0 forever is the same lie as the stock photos.
- ⚠ Settle before the Buy path: `schema.md` lists `narrative_tickers.mint_ca`; `lib/types.ts` asserts there is no mint column.
- New CSS: `.ptk` / `.pthd`, `.pd-rank`, `.newpill`, `.storydot`, `.lead-late`, and a new `.pbtn.buy` — `.pbtn.trade` already means Create Coin in the rail.
- The sparkline lateral needs `and unavailable is not true` with views carried forward across gaps, or a deleted-then-restored post draws the exact one-step shape we flag as bought reach.

Build order

1. **Migration only.** All of the above, the snapshot index, drop `replica identity full`. **1a** norms pass over raw snapshots; **1b** `post_heat` backfill reading 1a's norms, replayed in continuous time against real `captured_at` deltas so backfill and live accumulator agree exactly. That equality is the test.
2. **Coinability classifier + enum.** A column and a model call, not a dependency — and the one thing deciding whether a row may exist. Nothing below goes on a public route until it lands.
3. **Offline control.** Seven days scored with HN's `(P-1)/(T+2)^6` against PostHeat, judged on whether the top-10 at t held posts whose stories produced a $\geq$ \$100k coin after t . If HN wins, steps 4–5 shrink to one line.
4. **Pipeline scorer.** EWMA, Gamma-Poisson LCB for views and reshares, the five O features with the TikTok exemption, lifecycle hysteresis, censored entry. Fixes the cold-start null at `velocity.js:50`. Pure functions, tested, no mocks.
5. **commit_board()**. Heat 60s, commit 20s with swap/enter/exit/pin/move plus author cap, story cap, phash dedup; norms hourly. Verifiable from SQL before a pixel exists.
6. **Static board.** ISR at `revalidate = 2`, hydration query, eleven columns, responsive collapse. Already a real product: a board that reranks every two seconds beats a feed that doesn't rank.
7. **Media.** Three-case thumbnail, source chips, TikTok hashtag treatment and DERIVED subject, deletion of the fabrication chain.
8. **Control strip.** Filters, sorts, thresholds, SHOW RISKY, COLLAPSE BY STORY, persistence, gate disclosure.
9. **Realtime.** Broadcast `board:posts`, 500ms coalescing, value-update map, tick animations, new-posts pill, freeze-on-hover, staleness states.
10. **Action resolution.** Story collapse, the safety-gated count, fee and slippage disclosure on both paths, promotion pulse and retraction demotion. Depends on `coin_matches`.
11. **Rail posts mode**, then the **detail sheet** with score decomposition, then the **compare surface** for $n \geq 2$ — mints ordered by score, `first` badge on the earliest, per-coin safety state, per-coin Buy each with its own confirm.

02. Live Feed — the coins lane, and Trending

What it is

A ranked terminal board answering two questions at once: what is moving right now, and where did it come from. The second half is what nobody else renders — every row carries a STORY column in the slot DexScreener and Pump.fun spend on a fake sparkline. Trending is the slower companion: a 24-hour survey of what you should know about, not a second copy of the board.

How it looks

Content column plus the 460px rail, rail gone below 1040px. Lane tabs, then a band strip, then the table. Nothing centred, nothing spacious.

The band strip is a segmented control (`.chips` , not `.trend-sub`s — four bands plus counts plus a toggle need an active pill): **Fresh** · **Graduating** · **Live** · **From a Story**, each with a mono count. `Fresh 38 · 6 hidden` — the hidden count is rows the risk gates removed, and it is always stated. `From a Story 7 · 4 unverified` , with a cyan dot when that count moved in the last 60s. Beside it, **Show risky**, a loud switch, off by default; it relaxes a hard gate, so it does not live in a drawer. Route is `/feed?lane=coins&band=fresh` — lane and band both in the URL, so a trader can send a colleague exactly what they are looking at.

Four bands, never one list. A 3-minute \$8k curve token and a \$40M AMM token are not comparable quantities, and each band gets its own columns — four structural dashes per row across a whole band is a table admitting it was built for the wrong data.

BAND	COLUMNS
Fresh (<code>curve_pct < 50</code> , age ≤ 6h)	# · COIN · STORY · AGE · PRICE · MCAP · CURVE · VOL · HLD · RISK · action
Graduating (<code>curve_pct ≥ 50</code> , not migrated)	same, CURVE promoted to slot 4 as the sort key, LIQ replacing MCAP
Live (migrated / has a pool)	# · COIN · STORY · PRICE · 5M · 1H · 24H · VOL 24H · LIQ · MCAP · HLD · AGE · RISK · action
From a Story	grouped by story, not by coin — below

is dim mono, not sortable: rank is the output of the sort. **COIN** is a 32px logo, symbol 13.5px/600, name muted beneath, socials in fixed order, lime NEW badge for 300s. **AGE** renders `4m / 2h / 3d` , computed at render, and goes full ink under 60 minutes — on a board selling earliness, age is a headline number, not chrome. **PRICE** is `$0.0000123` at 7dp below a cent, flashing lime/red for 850ms on change. **MCAP** is exact even at four minutes: pump supply is a fixed 1e9, so mcap = price × 1e9, no vendor involved. **CURVE** is a 44px cyan track with the percentage beneath, going lime above 80% so about-to-graduate is findable by peripheral vision. **VOL** is 5-minute on Fresh, 24-hour on Live; under five minutes old it shows volume since mint with the elapsed window as an 8px superscript (`$4.2K` over `3m`). **HLD** goes amber under 25 and renders `>5,000` when the account scan truncates, never a fabricated exact figure. **5M / 1H / 24H** are signed one-decimal `.chgpill`s; a window longer than the coin's age renders `—` with `title="coin is 41m old"` — never a `0.0%` pill, which reads as flat rather than undefined and is the most misleading render available here.

STORY has six states. CONFIRMED: 16px thumb, title at 12px, cyan `.lead-badge.sm` — cyan, not lime, because every earliness affordance in the app is cyan and lime means gain. MULTI: the same plus `· 1 of 3 coins` ; a coin whose story has siblings says so wherever it appears. UNSURE: thumb at 60% behind a dashed ring, muted title, amber `UNVERIFIED` , no lead chip, no cyan. ADOPTED: amber `ADOPTED` plus the coin's true age, because the earliness claim is inverted there. MATCHING: `matching...` in dim mono with the existing pulse. NONE: the literal string `no story` in dim mono — no border, no chip, no box. On Fresh that is 85–95% of rows and any badge treatment turns the column into static.

The lead chip has **three branches, because negative lead time is common**: discovery latency is p50 ~12 minutes and a fast deployer mints four minutes after the post. Measured against `t_crypto = min(t_mint, t_ct) : +18m vs CT` in cyan at ≥ +2m, `same minute` in plain dim inside ±2m, `-8m vs CT` in amber below — we were late on that one, said plainly. Beside it, when a story is still unpicked by crypto Twitter, the `.timing-pill.timing-early` **Early** pill. That is the only earliness question a trader actually asks, and it is live rather than historical.

From a Story is grouped by narrative — one row per story. Three confirmed mints under one story is one row reading `See the 3 coins` , expanding in place to a sub-table ranked by match score, then earliest mint, then liquidity. Not three rows with three identical titles and three Buy buttons. The action is the settled coin-count function: **0 → Create Coin**, drawn as the existing `.tk.pend` row (dashed icon, `no coin yet` , `awaiting launch` , numerics `—`) — the head start while it is still spendable, which belongs here and not only in retrospect as a chip on a coin that already exists; **1 → Buy**

\$TICKER; 2+ → See the N coins. The band adds a LEAD column and widens STORY at COIN's expense: here the story is the subject and the coin is the instrument. Clicking `4 unverified` reveals the abstained candidates inline under a dashed separator in the UNSURE treatment — labelled unverified, primary still Create, **no Buy rendered on those rows at all**. Abstain never means hide, and it never means Buy either.

On the three coin-subject bands the action is a cyan `Buy` , but it is navigation: it opens the coin page with the tradebox focused, where the quote is fresh and the fee is disclosed before any confirm. The floor is real — the Jupiter integrator fee has a 50 bps minimum, so 0.5% per trade is the cheapest number that can ever appear.

RISK is a two-part chip, 62px, reusing `.risk-pill` : worst discriminating signal left, fixed obtainability denominator right — `T10 78 · 4/9` , `H 11 · 4/9` , `NO SELL · 4/9` , `DEV ×7 · 5/9` . Nine signals exist, we check four, five are not obtainable: dev holding, sniper %, bundle % and insider % need first-slot analysis we are not building, and a missing chip reads as "fine." A lime `✓ 5/9` is unreachable until creator history has a verdict for that wallet; before then the chip reads `? 4/9` . A clean tick can never be manufactured by absence of data — that is the entire design of the cell. Hover opens a 260px popover listing all nine, the five unobtainable ones named as "not available at launch" rather than omitted.

Fit budget. Live's 14 columns need ~1750px with the rail, ~1290px without. On a 1440px laptop with the rail the table gets ~928px and renders **9 of 14 columns**, the container query dropping HLD → 24H → LIQ → VOL → AGE. Container query on the table's width, not the viewport — the rail vanishing at 1040px gives back 460px and a media query gets the direction backwards. Every numeric track has an explicit minimum (PRICE 78px, change pills 66px, MCAP/VOL/LIQ 72px) so the grid drops columns instead of squashing `$1.23M` into 35px. COIN, STORY, PRICE, MCAP, RISK and the action never drop; STORY surviving to the narrowest breakpoint is not negotiable, because without it this is DexScreener.

How it works

```
V5      = vol_buy_5m + vol_sell_5m
accel   = (V5 * 12) / max(V1h, 1)           1.0 = steady state
breadth = clamp(log1p(traders5) / log1p(50), 0, 1)
netBuy  = net_buyers_5m / max(traders5, 1)  Jupiter's own net-buyer count
org     = (organic_buy_5m + organic_sell_5m) / max(V5, 1)  only when age ≥ 10min

quality      = (0.30 + 0.70*org) * clamp(1 + 0.5*netBuy, .6, 1.5) * safetyMult
quality_cold =                    clamp(1 + 0.5*netBuy, .6, 1.5) * safetyMult
coreHeat     = log1p(V5) * breadth * min(accel, 6)^0.5 * quality
```

`CoinHeat_fresh = coreHeat / ((age_min+20)/20)^0.9 · CoinHeat_live = coreHeat · CoinHeat_story = coreHeat × (0.5 + 0.5×narrative_heat) × (t_ct IS NULL ? 1.0 : 0.35)` . Graduating ignores heat and sorts by curve progress, which is the question that band exists to answer. The cold path is mandatory, not an edge case: Jupiter's organic score is unreliable under ten minutes — precisely our window — so `quality_cold` never reads it. The 0.35 CT-pickup factor is the thesis in the formula: the moment crypto Twitter finds a story, our edge is gone and the coin should fall off the board even while it is still climbing.

Four things stop a \$1.8B token owning rank one. **Band partition** — it is in Live, structurally never in the same ordered list as a curve token. **Mcap and liquidity are not terms** anywhere; mcap is a display column and a gate. **Acceleration is self-normalised** — `V5×12 / V1h` compares a coin to its own last hour, so being big earns nothing. And the **Fresh age penalty** (1.87× at 20m, 3.48× at 60m, 7.94× at 3h) turns that band over completely in about forty minutes.

Story attachment does **not** multiply rank in the other three bands. It is a filter and a tiebreak only. If it multiplied, the incentive would be to game the matcher — and the matcher is the 0.97-precision-critical component. Keeping story off the multiplier keeps the attack surface off the correctness-critical path.

Hard gates hide a row unless Show risky is on: mint or freeze authority active, Live liquidity under \$3,000, `sells_5m = 0 AND buys_5m > 30` (honeypot proxy), `mint_time IS NULL` , and top-10 concentration above **85% on Fresh / 60% on Live** — band-dependent, because a flat 60% gate empties Fresh on day one, when a three-minute-old token has nine buyers and is legitimately concentrated. Two facts decide whether the column works at all: the **bonding-curve PDA must be excluded** from top-N and holder count, or every pre-graduation token reads 80–95% concentrated; and RugCheck's authority fields must go tri-state, since the wired endpoint currently treats `mintAuthority = null` as revoked-and-safe, so an unreachable vendor passes every row. Null fails closed and increments the unknown count.

`safetyMult` multiplies down for concentration (>60% ×0.35, >40% ×0.70), a >25% top-1 holder (×0.50), under 25 holders (×0.60), the honeypot proxy (×0.20), creator history (two prior rugs ×0.30; 20+ mints with a sub-\$15k median peak ×0.55), thin Live liquidity (×0.40), plus two cheap wash detectors — trade-size Gini under 0.25 over 30+ trades (×0.5), and >60% of trades at exactly 0.1/0.5/1.0 SOL (×0.6). Clamped to [0.15, 1.0]. Creator history is free: the PumpPortal socket already running as the earliness clock carries the creator wallet on every create event, so persisting it gives prior mints, median peak mcap and rug count (peak ≥ \$30k, current ≤ 10% of peak, age ≥ 24h) after ~14 days at zero marginal cost.

Cadence. The rank commit is 20s, so the data behind it cannot be 5-minute grain — `accel` , the term doing the actual work, would update twelve times slower than the board claims. Poll Jupiter every 30s for the ~120 mints on any board ($\approx$ 4 RPS batched on the \$25/mo Developer tier) and roll *down* to 5-minute grain for 48h retention; off-board mints stay at 5 minutes. Holders move slowly and get their own 5-minute Postgres-cached refresh, cutting RPC volume $\sim$ 15 $\times$. Staleness fires at 3 $\times$ each feed's nominal interval — $\sim$ 90s for prices.

Stability: `EPS_SWAP 0.08` , enter after 2 ticks, exit after 3, pin 90s, max 5 positions per tick, freeze-on-hover, new entrants above row 12 batched into a cyan `3 new` pill. Values update every tick from a `Map<mint, Row>` ; only the ordering array is held. The escape hatch covers **movement, not just entry**: a coin clearing a hard V5 threshold skips the dwell, ignores MAX_MOVE for that tick and jumps to its true rank with a NEW badge and a one-tick lime flash. Otherwise a coin entering at 40 and deserving 1 takes 160 seconds — the core claim spent on anti-jitter.

Trending keeps its settled weights — `0.40 $\cdot$ min(vol24h/mcap /3,1) + 0.25 $\cdot$ min(txns24h/4800,1) + 0.20 $\cdot$ min(max(chg24h,0)/85,1) + 0.15 $\cdot$ min(max(chg1h,0)/55,1)` , $\times$ `safetyMult` , recomputed every 5 minutes. Eligibility: `mcap  $\geq$  $30k, liq  $\geq$  $5k, vol24h  $\geq$  $10k, age  $\geq$  6h` — exactly where Fresh's ceiling ends, so no coin falls between the surfaces. Between 6 and 24 hours the 24h terms use since-mint quantities and the row carries a dim `partial windows` qualifier. The real boundary between the two surfaces is window length, not age: the board measures five minutes of flow inside a comparable band; Trending measures a day. Layout is `/trending` , no sub-tabs — Stories and Coins side by side at $\geq$ 1240px in a 1140px wrapper, stacking below that, eight rows each so both are visible at once instead of one being a scroll away. Coins header: `# | TOKEN | 24H | VOL | RISK | MCAP` , the TOKEN cell gaining a second line (10px cyan dot plus story title) on a confirmed match and omitting it otherwise. Not sortable — a sortable top-8 is a bad version of the board — with one interaction, a 24h/6h/1h selector re-scoping the change column.

States

STATE	WHAT THE USER SEES
Default	40 rows, 20s rank commit, values ticking live, reorders frozen while the pointer is in the table
Brand-new token, no market data	Pre-rank zone above row 1, max 5 rows, dashed top border, <code>•</code> in #, <code>..</code> in every numeric, a live <code>MINTED 8s AGO</code> chip. Selection among ~40 arrivals per 90s: story match by narrative heat, then creator median peak mcap, then arrival order, with a dim <code>and 38 more minted in the last 90s</code> . Unranked is not rank one
Zero trades at 90s	Dropped from the surface, not silently zero-ranked into row 40
Can't-get vs not-yet vs pending	<code>—</code> dim = unobtainable; <code>—</code> dim with <code>title="coin is 41m old"</code> = window not elapsed; <code>..</code> = expected imminently. Never 0, never 0.0%, never a green tick
Unverified match	Dashed ring, muted title, amber <code>UNVERIFIED</code> , no lead chip. Click opens the evidence popover, not the story — navigating asserts a claim we have not established. The popover routes to the coin page; any story page reached from an unsure cell mounts with Create as primary and no Buy control of any kind
Story with no coin	<code>.tk.pend</code> row in From a Story: <code>no coin yet</code> , <code>awaiting launch</code> , numerics <code>—</code> , action <code>Create Coin</code>
Multi-coin story	One row, <code>See the 3 coins</code> , expanding to a ranked sub-table; siblings elsewhere read <code>• 1 of 3 coins</code>
Negative lead time	Amber <code>-8m vs CT</code> , plainly. Inside ±2m, <code>same minute</code> in dim, no colour
Stale pipeline	Per-feed stacked amber lines with live counters (<code>prices 4m 12s • Jupiter degraded</code>). Only the price feed dims PRICE and VOL; Buy stays primary — it is navigation, and the quote is refetched at the destination where <code>#quoteNote</code> reads <code>board prices were 4m stale — quote refreshed just now</code> . Safety past 30m dims the RISK chip with an age suffix: <code>T10 78 • 42m</code>
Show risky ON	Gated rows appear with a red chip naming the gate: <code>MINT AUTH</code> / <code>NO SELL</code> / <code>T10 91</code> / <code>LIQ \$1.2K</code>
Wallet not connected	Buy renders fully live in cyan → coin page → tradebox with the Privy modal and <code>Connect wallet to trade</code> . Greying it out kills the connect funnel
Matcher abstaining	Dim line under the band strip: <code>matcher abstaining on 34% of candidates</code> — a corpus-staleness signal, not a failure, but visible rather than buried in ops
Matcher down	No verdict in 20 minutes → <code>matcher offline 22m</code> in amber replaces the pulse, which otherwise reads as activity rather than a dead component
Story withdrawn	Row flashes amber once and settles to <code>story withdrawn</code> , rather than silently reverting to <code>no story</code>
Band transition	A migrating coin flashes lime, animates out over 400ms, enters Live with a NEW badge. Not a rank move; the stability rules don't apply
Non-pumpfun launchpad	NULL <code>curve_pct</code> , so band is a CASE with an explicit else: Fresh while age ≤ 6h, Live once a pool exists. Never nowhere; CURVE shows a dim <code>DEX</code> badge
Empty band	<code>No coins under 50% curve right now.</code> / <code>Nothing approaching graduation.</code> / <code>No story-linked coins in the last 6 hours — the matcher abstains when it is not sure.</code>
Thin Trending	Only qualifiers, plus <code>only 6 coins cleared the floor in the last 24h.</code> Never padded to eight — a padded top-10 is the same lie as a hits-only track record
First run	One dismissible dim line per band: <code>From a Story: coins we traced back to a viral post before crypto found it.</code> First visit lands here — smallest, most explicable band, the only one whose rows carry a human reason to care
Loading	12 skeleton rows sized to the active band's template so nothing jumps

What it needs

The headline: **the coins lane has no table.** `narrative_tickers` is the only coin storage and its spine is `narrative_id`, so a coin with no story — 85–95% of this board — has nowhere to live. A missing entity, not a missing column.

New: `coins` keyed on `mint` (symbol, name, logo_url, decimals, creator_wallet, `mint_time timestamptz`, launchpad, curve_pct, migrated, pool_address, socials, first_trade_price, first_seen_at). `mint_time` replaces the stored `age_min` scalar, which is wrong the second after it is written; precedence is the PumpPortal create event, then `min(pairCreatedAt)` across *all* Solana pairs, NULL only if both are absent — the best-pair method reports 418 days for a token that is 647 days old. `band` is generated, not stored. `coin_snapshots`: there is no price column anywhere in the schema today and no coin time series at all, so every window and the whole formula are currently uncomputable — price_usd, mcap, liquidity, 5m/1h/24h buy and sell volume, buys/sells/buyers/sellers/**net_buyers**/traders, organic

fields, holder_count, top1_pct, top10_pct, captured_at. `coin_matches` : match state has no home, since `canonical` and `first_deployed` are booleans and cannot express UNSURE — mint, narrative_id, match_state, relation, score, the five-channel evidence vector for the popover, matcher_version, `UNIQUE(mint, narrative_id)` . Lead time is computed at render, never stored. Log every confirmed and unsure row with its vector; that is the eval set growing itself.

Also new: `narratives.promoted_at` , set once forever, NULL for pre-migration narratives — which renders no lead chip and excludes them from From a Story rather than fabricating a time; `narrative_tickers.source` ('cashtag' | 'llm' | 'mint_in_post'), whose absence is the root of the \$KANG defect where an LLM-invented ticker can become canonical; `creator_history` ; `coin_board_ticks` (band, captured_at, mint, rank, heat, plus p95 heat per band for the presented-heat normalisation); and `coin_safety` , a Postgres RugCheck cache, because the current 300s in-memory cache is per-lambda and effectively always cold.

Two queries per band, not one: rows via a `LEFT JOIN LATERAL` collapsing to one match per mint (confirmed before unsure, then score) with a sibling count for the MULTI state, plus a `LEFT JOIN` to the latest snapshot so pre-trade coins survive; and a second aggregate for the gated-row count. Delta push via Supabase Broadcast keyed by mint — values apply immediately, positions on the tick.

Two things stay flagged as unverified. Whether excluding the curve PDA yields a top-10 distribution with real spread needs checking against 20 live pre-graduation tokens before the RISK column is designed around it. And the graduation threshold is SOL-denominated (~85 SOL) — read it from the deployed program once per deploy, never a hardcoded USD figure. Separately: whether the stories lane stays merged into Live Feed or becomes its own tab is still open with you; this is written merged.

Build order

- `coins` , keyed on mint, backfilled from `narrative_tickers.mint_ca` . Nothing else is possible first.
- `mint_time` from the create event with pair-minimum backfill; delete `if (!pair.pairCreatedAt) return 0;` at `lib/token-lookup.js:70` and fail closed instead.
- `coin_snapshots` on the Jupiter Developer tier, 30s board polling rolled down to 5-minute grain.
- Fresh band only**, ranked by `CoinHeat_fresh` with `quality_cold` . No story column, no risk chip — a shippable terminal that validates the ranking first.
- Risk chip and safety multiplier** from the already-wired holders and safety endpoints, including the curve-PDA exclusion, the RugCheck tri-state fix and the Postgres cache. Without the exclusion the column is uniformly red.
- Graduating and Live**, with `curve_pct` from the PumpPortal trade stream's reserve fields — correct by construction; RPC reads only for stream-missed mints.
- `coin_matches` + `promoted_at` + `source` , shipped with the day-one matcher fixes. Scope it honestly: 40+ lines across three files plus a migration and a backfill.
- The STORY column, all six states, and the story-grouped From a Story band.** This is the step that makes the product differentiated; everything before it is a competent DexScreener clone.
- First-trade capture** from the trade stream, written once and immutably — only then does Δ MINT ship. Until then Fresh shows VOL-since-mint with the elapsed-window superscript, honest at any grain.
- Trending** — existing mini-tables plus the story line, risk column and 6h floor. Deliberately last: a digest of untrustworthy data is worse than no digest.
- `creator_history` , shipping dark and accumulating ~14 days, after which the DEV signal turns on and the lime tick becomes reachable.
- Realtime** — Broadcast, the five stability constants, freeze-on-hover, the new-entrant pill, the movement escape hatch. Polish over a board that must already be correct.

03. The Story page

What it is. `/story/[id]` is where a user settles two questions — "is this real?" and "am I early or late?" — and then takes exactly one action. It is a routed page, not a modal: the lead-time claim is unfalsifiable without a URL a stranger can open and check, and the discussion layer lives here.

How it looks

Two columns on `#060607`. Left is evidence, one document scroll. Right is the persistent rail, which on this route opens on a fourth tab mode — **Discussion** — beside the existing tape modes. The rail does not disappear here; a trader sits on this page longest and no terminal hides the tape on its analysis view. With zero comments the rail stays on the tape and the invitation becomes a 52px "Add your take" strip pinned under the coin section, where a first commenter will actually see it. Full-width amber bands sit above everything when they apply: stale pipeline, then coinability `no_create`.

Head, 52px: a back chip carrying the real referrer (`← Live feed`, `← Search`, `← You`), the title in Clash Display 30px clamped to two lines, then `☆ Follow`, a copy-link button that flashes lime for a second, and a chip reading `live` or `stale 34m`.

Verdict bar, 84px, five cells with hairline dividers. A readout, not a control.

CELL	SHOWS
LEAD TIME	Our record, mono 24px: <code>+31m</code> lime · <code>-12m</code> red · <code>+2h14m</code> cyan · <code>unmeasurable</code> amber · <code>not promoted</code> dim
YOUR CLOCK	<code>promoted 4h12m ago · 2 coins since · you are not early</code> (amber) / <code>promoted 6m ago · no coin yet · you are ahead</code> (lime)
STATE	Dot + lifecycle word, sub <code>posts/5min 14 → 22</code>
REACH	<code>2.4M</code> · <code>1.2K/min</code>
COINS	<code>1 confirmed</code> / <code>0 confirmed · 2 unverified</code> , coloured to match the section below

Those first two cells are the correction that matters most. `+31m` is a fact about the detector, frozen at promote; it says nothing about whether the head start still belongs to the person reading. YOUR CLOCK is `now - promoted_at` against `count(confirmed)`, and it is the only number that answers "am I early." They never share a component, and the sticky action bar never renders `+31m`.

Source post, 132px: 96px media with a `from video` / `no media` chip, then handle, followers, platform, clock time, three lines of text, and `2.4M views · open post ↗` in cyan mono. The card links to the real permalink from `platform_post_id`; if that id is missing no link renders and a footnote reads `permalink unavailable for this post`. We never synthesise one.

Virality grid, 62px, seven cells — mono value, 9.5px uppercase label, 30-minute delta: VIEWS `2.4M` `+310K/30m` · REPOSTS `5.4K` · LIKES `115K` · REPLIES `8.2K` · AUTHORS `14` `+4/30m` · PLATFORMS (marks, no number) · AGE `4h`.

The coin section — the action, fully above the fold at ~574px on a 1440×900 screen. One panel shell in every state. A count chip in the header carries the state's signature so a zero never reads as an error: `NO COIN` dim, `1 CONFIRMED` lime, `0 CONFIRMED · 2 UNVERIFIED` amber. The primary is always bottom-right, 40px, min-width 168px; only the label moves.

No coin — the panel tints cyan, the only state that does. "No coin exists for this story." Prefill chips `$FERRY from post` (cyan) or `$FERRY suggested` (dashed — the only place in the product an LLM-suggested ticker may appear). Primary: **Create coin.**

One confirmed — a match chip in categorical words only, `matched · image + name · minted 4m after the post`. No percentage is ever rendered beside an actionable control. The row: 36px logo, `$HORN` + name + copyable truncated mint, MCAP `$799.6K`, LIQ `$41.2K`, AGE `2h`, then **Buy \$HORN**, which navigates to `/coin/<mint>`. Stories never buy; coins buy. An ADOPTED match shows its real age in amber and adds "This coin existed before the story. It was not made from it."

Two or more confirmed — a ranked table, `# | | Coin | Mcap | Liq | Age | Δ from post`, three rows and a `+2 more` that expands in place. **No per-row Buy chips:** N Buy controls in a table is the exact ambiguous state the one rule deletes. Rows navigate, nothing more. Default sort is `mint_time ASC` — "the first one made from this story" — because sorting by market cap systematically demotes the freshly minted derived coin, which is the coin the pipeline exists to find. `FIRST` (cyan) marks the earliest mint. `BIGGEST` (lime) marks row 1 only when `mcap[0] ≥ 2 × mcap[1]` and both

are non-null; otherwise nothing is badged and a line reads "No coin is clearly biggest — the top two are within 2×."

Printed beneath: `FIRST = made from this story before any other. BIGGEST = most money in it.` Primary: **See the 3 coins.**

Unsure — the hardest one. The panel is not dimmed and not greyed; grey reads as broken. Region 1 is full brightness and cyan-tinted exactly like the no-coin state: "No confirmed coin for this story." with an enabled **Create coin**. The page's centre of gravity stays a live cyan button, and that is what stops the state from looking like a failure. Region 2 sits below a dashed rule under an amber `UNVERIFIED CANDIDATES · 2`: "These mints share a ticker or a name with this story. We could not confirm they are related." Candidate rows carry an amber left bar, a placeholder glyph rather than the token's logo (a logo borrows credibility we declined to grant), mcap and age one contrast step down, a channel readout `ticker matches · name does not · image not checked · minted 3d after the post`, and one action: **View anyway**. Buy does not exist in this region at any size.

Fees sit in the section footer, always: `Insidor takes 50 bps on trades routed from here — the floor, not an estimate.` The create line reads "Creating costs about 0.021 SOL in rent and fees, quoted live. The fee split is not final." We do not print "you keep 50% forever" while that is undecided; when it lands, all three shares render together — `you 50% · the poster 40% · Insidor 10%.`

Momentum, 172px: a cyan filled views area with lime posts-per-5-min bars behind it — the acceleration signal the trigger actually uses, visible because views can climb while the rate collapses. Markers: PROMOTED cyan solid, FIRST CT POST amber dashed, FIRST MINT lime dashed. Beneath, the lifecycle word in Clash with a 3px coloured bar and then a mono readout, not a paragraph: `Ř 1.40 · v 0.52/min · g +0.020 · 22 posts/5m · 2 platforms`. The page keeps exactly two sentences of English — the lead-time claim and the outcome verdict — because both are falsifiable assertions and should read as such. Everything else tabulates.

Lead-time proof, 92px: four marks with the gaps on the connectors — `FIRST POST 14:02` —9m— `WE SAW IT 14:11` —+31m— `FIRST CRYPTO POST 14:42` —+3m— `FIRST MINT 14:45`, the lead connector 2px lime. Below it the claim sentence, two external permalinks, and always the provenance footnote: `measured against 41,208 mints streamed + 300 crypto accounts polled every 2 min · t_crypto = first of either.`

Then the **outcome band** when the story is finished, and the **posts table** last: `# | | Post | Views | Reposts | Δ30m | Age`, sortable, rows opening externally.

How it works

The primary is one pure, unit-tested function with no I/O:

```
confirmed = matches.filter(verdict == 'CONFIRMED' && mint_time && ticker && mint)
0 → Create coin      1 → Buy $TICKER      2+ → See the N coins
```

`unsure.length` is in the return value and nowhere in the branch condition. The `ticker` guard is load-bearing: ticker comes from `narrative_tickers`, which enrichment truncates and re-inserts every cycle, so a page load racing a cycle would render `Buy $undefined`. Ticker and name are denormalised onto `coin_match` at match time so the button never reads a table that can vanish underneath it.

Lead time is `t_crypto = min(t_mint, t_ct)`, `lead_min = (t_crypto - promoted_at)/60000`, with `promoted_at` written once and never recomputed. Negative renders `-12m` red, the marks reorder so crypto sits left of us, and the sentence is "Crypto got here first. The first crypto mention was 12 minutes before we promoted this." No hedging adverb exists in that path.

Gap detection has to be symmetric or the number is a lie. `mint_stream_gap` and `ct_poll_gap` are both written by a supervisor; if either overlaps `[promoted_at, t_crypto]` the verdict is `unmeasurable` and the story leaves the aggregate record. `+2h14m and counting` additionally requires both clocks green for the whole window — a live-incrementing cyan earliness claim generated by a dead poller is the worst failure available here. The existing `ct_pickup` boolean is a second, regex-based definition of the same event; one is canonical and the provenance line says which.

The momentum series is a rollup **table** in 5-minute buckets, written incrementally by the snapshotter with last-observation-carried-forward per post — not a matview over raw snapshots. Posts are sampled on a per-post geometric grid, so summing by minute makes the story's total drop whenever fewer posts happened to snapshot. `v`, `v_peak`, `g` and `Ř` all read off that series, so COOLING would fire on a sampling artefact and the page would print "window narrowing" over a story that is accelerating.

Staleness is per-worker with a work-done check, not `max(ran_at)`. The one outage that has actually happened — X ingestion dead while the snapshot and trends crons fired normally with zero results — reads as `live` under a global max. So: stale if the ingest worker's last cycle is over 10 minutes old, **or** the last six ingest cycles ingested zero, **or** the last three snapshotter cycles wrote zero. The band names the dead lane: "X ingestion last produced a post 34 minutes ago."

Realtime does not use `postgres_changes` on `comment`. That stream ignores the read-path status filter and would broadcast every address-gated and silently dropped row to every client — delivering the spammer’s contract address faster than an unmoderated feed. Comments broadcast from the insert handler on a server-owned channel, sanitised row only, after the status check.

A second channel, `coin:<narrative_id>`, carries the transition the product exists to occupy. The button does **not** mutate under a travelling finger: it freezes, and a 44px cyan interstitial appears — "A coin was just made for this story. \$HORN, 40 seconds ago." with an explicit **Show it**. Demotion uses the same pattern in reverse. `/create?story=<id>` is gated server-side on a fresh `count(confirmed) = 0` at submit, so nobody pays to duplicate a coin the page was hiding.

Discussion sorts `snap_views ASC, created_at ASC` — earliest means *written when the story was smallest*, not oldest by clock. Each comment carries a stamp frozen at insert, one line with ellipsis: `posted when 340K views · no coin yet · 40m old · early`. Three facts at zero coins, four when a coin existed; `$0 mcap` reads as a bug, not a fact. Every snap column including `snap_early` is written once and guarded by a BEFORE UPDATE trigger — computing `early` at render against live views would let the badge drift monotonically in the author’s favour as the story grows.

The position badge is read once server-side at write time and stored as raw amount, decimals and mint: `holds 2.1M $HORN`. Not SOL — native SOL is unrelated to the coin, and SOL-denominating a token position needs a price quote we do not fetch. A failed read renders `position unknown`, dashed, never `no position`.

The address gate blocks **every** base58 run of 32–44 characters, including this story’s own mints. There is nothing a user can say with a contract address that they cannot say with `$KANG`, which is already auto-linked — and letting UNSURE mints through was the page’s highest-value skill vector, since landing a copycat in the abstain band is the expected path, not an exploit. The run is replaced inline with the ticker chip and the comment posts. The client check is a courtesy; the server runs the same function and is the enforcement, shipping in the same release as the composer.

Outcome classification runs nightly on `now - promoted_at > 48h` regardless of lifecycle: gap → UNMEASURABLE, 0 confirmed → NO_COIN, `lead_min < 0` → LATE, `peak_mcap ≥ 250K` → HIT, else DUD. A story whose snapshots stop terminates as `lifecycle_reason = 'starved'` after six quiet hours — otherwise `v` goes NULL, neither DEAD predicate fires, and misses never enter the record while hits do. A monitor holds `count(outcome IS NULL AND promoted_at < now - 72h)` at zero.

The share card branches on state: `no_create` emits the title, "A story on Insidor" and a generated placeholder — no lead-time claim, no coin count. Negative, unmeasurable and not-promoted stories never emit a boast. Images are generated from our own palette, never hotlinked.

States

STATE	WHAT THE USER SEES
Loading	Skeletons for the verdict bar and source post; the coin section a 96px pulsing block with its label already in place; discussion on its own boundary so a slow query never blocks the evidence
First visit	Cell 1 gains "before the first crypto account mentioned it"; the count chip gains a one-line gloss; the Earliest definition renders inline instead of on hover. One story, dismissed on any interaction
New token, pool unindexed	<code>indexing...</code> with a pulse in MCAP/LIQ, <code>NEW</code> badge, Buy enabled
New token, bonding curve live	<code>— · no trades yet</code> , "you would be the first buyer" — enabled only after a real Jupiter quote returns a route
New token, no route	Primary becomes a disabled-with-reason line: "Not tradeable yet. Watch it." plus Follow
Unverified match	Amber region, <code>View anyway</code> only. <code>/coin/<mint>?unverified=<story_id></code> names the referring story; the warning belongs to the (story, mint) pair, never the mint alone
Coin arrives / match demoted while open	Button freezes, cyan interstitial, re-render only on click
Story with no coin	Every stamp reads <code>no coin yet</code> , every badge <code>no coin to hold</code> , <code>· early</code> common. A cyan divider marks where the first coin appeared, once it does
Negative lead time	<code>-12m</code> red, timeline reordered, "Crypto got here first." The outcome band later reads "We were 12 minutes late." Every region agrees
Unmeasurable	Amber dashed: "The mint stream dropped from 14:31 to 14:52. We will not claim a number we could not verify."
Not promoted	<code>not promoted</code> ; the timeline is replaced by one line. <code>created_at</code> is never substituted
Already hot	Dim dashed chip: "We first saw this post already large. Excluded from our lead-time record."
Stale pipeline	Amber band naming the dead lane; REACH stops animating and gains a <code>stale</code> chip. Numbers still shown, all labelled
Match run stale, not absent	"Coins last checked 3h ago" — never a confident "No coin exists" that is merely old
Matching not yet run	"Checking for coins...", no button, times out to the no-coin state after 60s with a footnote
Coinability <code>never</code>	404. Not a hidden page, not an empty state
Coinability <code>no_create</code>	Full page, discussion open, amber band: "...You can discuss it here. Insidor will not help you make a coin from it. Coins that already exist are listed below and we earn a routing fee if you trade them." No primary button — not a disabled one
Wallet not connected	Fully readable, Follow works, composer collapses to a 40px connect row, the primary keeps its label and opens Privy first
Insufficient SOL	The Create primary carries a sub-line with the shortfall — before the sheet, not at signing
Never traded	Pre-stamp reads <code>no position</code> as a neutral uncoloured fact, not a judgement
Position read failed	<code>position unknown</code> , dashed. A chain read never blocks a post
Blocked / held comment	Struck through with a red note, visible to its author only. Everyone else sees nothing
Empty discussion	"No takes yet." plus the pre-stamp as invitation, inside the collapsed strip under the coin section
Merged while open	"This story was merged into another cluster" + link to the survivor; the comment channel closes rather than writing to a dead id
Post deleted upstream	Red left bar: "This post has been deleted. Last seen at 2.4M views, 14:41."
Momentum series empty	"Too new to plot — first snapshot at t+4m"; the state word still reads "Just spotted"
Reduced motion	Tick animations, flashes, arrivals and the 60s counter resolve instantly to their end state
Not found	"No story with that id. It may have been merged into another cluster." Merged ids 301 to the survivor

What it needs

Usable now: `narratives`, `narrative_posts`, `narrative_tickers`, `post_snapshots`, `post_meme_scores`, `worker_cycle_log`. Note that `distinct_authors`, `author_velocity`, `ct_pickup` and `subject_entity` **do exist** (`worker/schema-replication.sql`, written by `cluster-persist.js`) despite being absent from `docs/schema.md`; `narrative_tickers.mint_ca` has a migration and a writer but its application to the live catalog is unverified — and the coin query, the one rule and the address gate all sit on top of it. Regenerate `docs/schema.md` from the live catalog before step 1 and re-derive this list.

Missing: `coin_match` (`narrative_id`, `hint`, `verdict`, `relation`, **`ticker`**, **`name`**, `per-channel scores`, `strong_channels`, `mint_time`, `delta_min`) · earliness columns on `narratives` (`promoted_at` with an immutability trigger, `t_mint` , `t_ct` , `first_crypto_post_url` , `first_crypto_handle` , `first_post_at` , `earliness_eligible`) · `coinability_tier` and `coinability_reason` · the social layer (`app_user` ; `comment` with the snap columns plus `snap_early` and the three position columns; `comment_vote` ; `holding` , without which `sold` is indistinguishable from "never bought") · the 5-minute rollup table plus an index on `post_snapshots` (`post_id`, `captured_at`) , verifying `captured_at` 's real type first · `mint_stream_gap` and `ct_poll_gap` · outcome columns (`outcome` , `lifecycle_reason` , `peak_mcap` , `current_top_mcap`) with `ticker_mcap_series` sampled every 5 minutes for 24h · `narrative_tickers.source` , so a suggested ticker can be a Create prefill and never a coin. `narratives.search_series` is the fabricated sine wave and must never render here.

Build order

- Schema.** Regenerate the schema doc from the live catalog, then one migration for the earliness and coinability columns. Backfill `promoted_at` only where recoverable; render "not promoted" everywhere else. Ships nothing; unblocks everything.
- Route and evidence, read-only.** Head, verdict bar (LEAD TIME reading "not promoted" until step 3), source post, virality grid, posts table, branched OG metadata. Delete `renderNarrativeModal()` and repoint every narrative click at the route.
- Momentum and lead-time proof.** The rollup table and snapshotter write, lifecycle with hysteresis and the starved terminal, the chart, the mono readout, the four-mark timeline including negative and unmeasurable — and both gap tables in the same PR, because without them every lead time is silently inflated.
- coin_match and the coin section.** Pipeline writes, denormalised ticker, `min(pairCreatedAt)` across all pairs, fail closed on null `mint_time`. Then the four states, the evidence popover, the quote probe and the fee footer. Until this lands the page can only show NONE or ONE — say so in the PR, do not fake it.
- Discussion.** `app_user` + `comment` with every snap column and its trigger, the insert handler, the composer with its pre-stamp, and the address gate client and server. Stamp and position badge ship here too — snap columns are unrecoverable after the fact.
- Moderation ladder** beyond the gate: impersonation hold, new-wallet limit, velocity drop, speed limit with countdown, report menu, author-only visibility.
- Realtime:** server-owned comment broadcast, the `3 new ↑` pill, and the coin channel with freeze-and-interstitial in both directions.
- Outcome band:** mcap sampling, the nightly classifier on the 48h rule, the band, and Search's Finished chip defaulting to All.
- Votes and one level of replies.** Last on purpose — the stamp is the better ranking signal and votes are farmable. Ship the stamp for eight weeks before deciding whether the vote button earns its place.

One thing stays open: whether stories keep their lane inside Live feed or graduate to their own tab. This page is unaffected either way — only the back chip's default label changes.

04. The Coin page — chart, swap, trade

What it is

One coin, told honestly, and the only surface in Insidor where money moves. Chart, stats and swap box are table stakes — Axiom and Photon render all of it. The job only this page can do is prove this coin exists because of a story we caught first, say by how many minutes, and refuse to sell you anything on a claim we do not stand behind.

How it looks

060607 page, max-width 1560: a wide main column beside a 356px right rail sticking 72px under the header.

Main column, top down. **Identity header:** 52px logo, `$FERRY` in Clash Display 27px, token name, a `new` chip under 10 minutes old, a `live` dot while page data is under 120s fresh, the mint as `4mZq...8xPq` that copies on click and flashes lime, socials; right-aligned `$0.000221` in mono 26px with the 24h change in lime `#9BF03C` or red `#FF5C6E` .

Then the **origin band** — the reason the page exists. Not a hero card: a ranked row in the terminal's own grammar, ~60px tall, hairline column headers, right-aligned mono, the whole row a button to the story.

CELL	SHOWS
thumb, 42px	the post's media
story, flex	<code>@lasthorn · 2h ago</code> at 10px uppercase --dim; <code>Harbor horn goes viral in Bergen</code> in Clash Display 13.5px; the post text curly-quoted at 72 chars in mono 11.5px --muted
SPOTTED, 92px	<code>14:02</code> — a literal clock time. Relative-only is unfalsifiable; an absolute time is what a sceptic checks against the permalink
CRYPTO, 92px	<code>14:33</code> , or <code>—</code> when nobody in crypto has posted
LEAD, 108px	<code>+31m</code> mono 22px cyan, sub-label <code>EARLY VS CRYPTO</code> . Beneath it in mono 10.5px, the reader's <i>live</i> lead: <code>your lead now +18m</code> counting down, flipping amber to <code>crypto caught up 7m ago</code> , red past 30m. The big number is Insidor's record; the small one is where the reader actually stands
matched, 76px	a categorical chip <code>matched · image + name</code> , opening the evidence popover

That popover is 320px: post media beside the token logo, chips `image ✓` `name ✓` `ticker ~` `mint-in-post —` , one line `Minted 4 minutes after the post — 14:04 vs 14:00` , footer `Matched by Insidor · Report a wrong match` . **Never a numeric score** — "73% match" beside a spend button makes the user do their own thresholding, which is what the abstain band exists to prevent.

Under the band, in the main column and not the rail, the **sibling strip:** `2 MORE COINS FROM THIS STORY` , rows of logo · \$TICKER · mcap · age, the earliest tagged `earliest` in --dim and the deepest book tagged `deepest` in cyan. Two neutral facts, no verdict word. It renders at one or more confirmed siblings and lists confirmed ones only — an unsure candidate here would read as endorsed.

Then the **stat grid:** eight hairline-divided cells — `5m` `1h` `24h` (signed %) · `Mkt cap` · `Liquidity` · `24h vol` · `Holders` · `Age` . Prices from the DexScreener pair; Holders from Helius RPC, which needs no pair; Age from `coin_mints.minted_at` , ticking live as `0:47` under an hour.

Then the **chart card**, 468px: a 40px head with the mono price and a `DexScreener` tag over a 440px embed of the *pool* address — never the mint, which resolves to the wrong pair, usually USDC. Then tabs: `Trades` (default, 10s poll, your fills highlighted cyan, click a wallet to filter), `Top traders` (only when a ranked trader returns), `Holders` , `Info` .

Right rail: **position card** (only when you hold), **trade box**, **safety box**.

The position card leads with dollars — `$179.60` mono 22px, then `+$38.40 (+27.2%)` , never a bare percentage — then `Qty 231,402 · Avg paid $0.000174 · Now $0.000221` , then the sentence nobody else can print: `You bought 41 minutes before crypto traders did.` The negative ships in identical weight: `Crypto traders posted 12 minutes before you bought.`

The trade box: Buy/Sell toggle; amou^utt field in mono 20px with presets `0.1 | 0.5 | 1 | 2 | MAX` (MAX = balance – 0.02 SOL for network fee, Token-2022 rent and retry headroom); a SOL⇌USD toggle defaulting to USD in Simple, SOL in Pro; a `YOU RECEIVE` panel reading `231,402.55 FERRY` at mono 19px over `$47.20 · 1 FERRY = $0.000204`; then five recap rows at one uniform 12px mono weight — `Min received 228,120 FERRY · Price impact 0.42%` (lime ≤1.5%, amber >1.5%, red >5%) · `Route dflow · Network + priority ~0.0009 SOL · Insidor + Jupiter fee 0.60% · $0.28 ⓘ`. All five come from the order response; nothing is a literal.

The fee row is the same colour, size and weight as its siblings — no badge, no box. The ⓘ opens: `Insidor fee 0.50% $0.24 / Jupiter platform 0.10% $0.05 / Total 0.60% $0.29 / Jupiter keeps 20% of the Insidor fee. / 0.50% is the minimum Jupiter permits on integrator fees — it is not a number we chose.` The confirm dialog repeats it uncollapsed before any signature. Ambient at equal weight plus pre-signature at full weight is disclosure without shouting.

Under the button, ordinary text rather than a red box: `If this goes to zero you lose $50. That happens most of the time.` A red box gets dismissed; a sentence gets read.

How it works

The page loads in one round trip: `tokens` left-joined to `coin_matches`, `narratives`, `story_clocks`, the top post and a `clock_health` row, ordered confirmed-first then score desc.

The numeral is `lead_time_min = t_crypto - promoted_at`, where `t_crypto = min(first matching mint, first crypto-Twitter post)`. `promoted_at` is written once at PROMOTE and never overwritten; it is NULL for every row predating the migration, NULL forces `earliness_eligible = false`, and those stories read `– / NOT MEASURABLE` forever. Backfilling it from `created_at` or `first_seen_at` would inflate every historical lead by roughly the pipeline's own latency — it would manufacture the number the product is sold on.

`FIRST` requires positive evidence of a live clock, not merely a null `t_crypto : gap_window = false`, a PumpPortal heartbeat inside 120s, a crypto poll inside 4 minutes. Otherwise LEAD reads `– / CLOCK DOWN` with a tooltip naming the blind sensor. Absent must never collapse into best-case.

Age uses `coin_mints.minted_at` — the same timestamp that produces `delta_minutes` in the band, so the two cannot contradict each other 150px apart. The fallback, `min(pairCreatedAt)` across all Solana pairs, carries a `pool age` micro-label, because a graduated pump.fun coin's pair timestamp is the graduation, not the mint. Missing returns null and renders `–`; unknown age must never present as brand new on a terminal that sells earliness.

The primary is a pure function of confirmed coin count on the matched story: 0 → Create, 1 → Buy, 2+ → Compare. **On an UNSURE match, Buy is not rendered at all** — not greyed, not gated, absent. The panel still quotes, the primary is cyan `Create the coin for this story`, and a 44px band under the origin band reads `The verified coin from this story is $FERRY →` when one exists. A coin with *no* story keeps a normal lime Buy: an unmatched coin carries no claim from us; an unsure one carries a claim we decline to monetise. Sell is never suppressed in any state.

Quotes: 250ms debounce → `POST /api/order` → Jupiter Swap V2, referral account and fee injected server-side from env, never from the client. TTL 20s, refresh every 15s while visible, suspended entirely while the confirm dialog is open — never re-quote under a finger. A refresh moving `outAmount` more than 1.0% against the user flashes amber and holds. `requestId` is single-use so every retry re-orders; the `trades` row is written at ORDERING with `request_id` unique, so the database enforces single submission across tabs. The displayed fee is frozen from the order that produced the signed transaction and re-asserted server-side against that requestId inside `/api/execute`; the actual charged fee is parsed post-fill into `fee_bps_charged`. Slippage defaults to Auto (Jupiter RTSE) under 30 minutes old — a fixed 1% on a live bonding curve reverts constantly, and every revert is a trust event.

Safety is four blocking checks on the **buy direction only**: mint authority active, freeze authority active, Token-2022 transfer fee or hook present, and no sell route (a reverse order call for ~1% of the intended out; buy route present with sell route absent is the honeypot signature, and worth more than the other three combined). Authorities come from Helius `getAccountInfo(mint, jsonParsed)` with three-valued logic — absent → `–`, null or the system address → revoked, pubkey → active. On pump.fun mints both are revoked by construction, so they collapse into one --dim line: `Mint & freeze authority — revoked by pump.fun on every coin (not a signal)`. Green ticks are reserved for checks that discriminate. Top-10 concentration is computed over circulating supply with the bonding-curve PDA, pool vault and burn address excluded; if the curve cannot be resolved the row reads `Top 10 holders — (bonding curve holds most of supply)` rather than a number that means nothing. Unknown never blocks, it discloses — and the block reason renders directly above the button, never in a side panel.

Selling is never blocked by a safety check; stranding a holder just moves the unfairness. A transfer-fee token discloses in amber in that same slot, and the fee is subtracted from `You receive` and added as a sixth recap row. `Take back what I put in` is solved against a real reverse order — binary-searched until net-of-fee proceeds meet cost basis, not `cost_basis / spot`, which is short by the fee and the impact — and labelled in the denomination it was computed in: `Take back my $141.20 (after fees)`. Below cost it stays visible and disabled: `Not enough value left to take back what you put in.` Hiding it hides a loss.

Cost basis never touches the browser's SOL price global: `/api/execute` captures SOL/USD server-side at submission into `sol_usd_at_trade`, and every PnL number derives from that column.

The band links to the story detail; whether stories keep a lane in Live feed or get their own tab is still open, and the link target is the same either way.

States

STATE	WHAT THE USER SEES
Brand-new, no pair (0–60s)	Chart card reads <code>Pool not indexed yet</code> with a live <code>mint 0:34 ago</code> counter — not an error. Price-derived stats <code>—</code> , never <code>\$0</code> ; Holders and Age live. The panel still quotes: <code>No chart yet — routed on the bonding curve</code> . Once a pair exists but history is thin, the chart mounts under <code>First trades — 4 minutes of history</code>
No route either way	Panel disabled, primary reads <code>Watch for a route</code>
Unverified match	Amber band, kicker <code>POSSIBLE ORIGIN · UNVERIFIED</code> , numeral replaced by <code>Unverified match</code> , a line reading <code>We are not confident \$KANG came from this story. Matched on ticker only.</code> + <code>Why? →</code> . No Buy; primary is Create; the verified sibling is promoted to a cyan band under the origin band
Match withdrawn (REJECTED)	<code>This coin was matched to Bergen Horn and the match has since been withdrawn.</code> No lead number, no sibling promotion; Buy behaves as unmatched
No story	40px neutral bar: <code>No story matched — this coin is not linked to anything Insidor detected.</code> + <code>Search stories →</code> . No earliness claim; normal Buy. The default for a pasted CA, so it must read neutral, not broken
Negative lead	<code>-12m</code> in red, label flips to <code>LATE VS CRYPTO</code> , sub-line <code>crypto posted first</code> , plus a footer line <code>Insidor was late on 31% of the last 200 detections — see the record →</code>
Lead pending, clocks healthy	<code>FIRST</code> in cyan, <code>NOBODY IN CRYPTO HAS POSTED YET</code> . Best possible news; must read as such
Clock down or gap window	<code>—</code> / <code>CLOCK DOWN</code> , tooltip naming the blind sensor. Never counted as a win
Adopted coin	Cyan chrome, kicker <code>ADOPTED BY THIS STORY</code> , sub-label <code>coin is 604 days old</code>
Two confirmed stories	The smaller lead is the headline; the larger is reachable as <code>also matched to Bergen Horn (+31m) →</code> . Score measures confidence, not earliness, and must not pick the flattering number
Stale pipeline	Amber strip: <code>Live data paused — prices last updated 4m ago</code> , <code>live</code> dot greys, panel notes <code>Quotes still live (Jupiter) — page metrics are stale</code>
SOL price unavailable	Every dollar slot <code>—</code> , never <code>\$0</code> . Fee falls back to <code>0.60% · 0.0026 SOL</code> , the amount field to SOL, the risk line to <code>you lose 0.22 SOL</code>
Decimals unknown	Panel holds in QUOTING until the mint account answers. A receive figure wrong by a power of ten is worse than a spinner
Checks pending (the common case)	<code>Checks pending — this coin is 47 seconds old.</code> We do not block on unknown — blocking every genuinely early buy deletes the product
Blocked (buy only)	Disabled with a red outline; the strip names it: <code>No sell route — we could not find a way to sell this</code>
Wallet not connected	Page fully readable and quoting; position card absent, button reads <code>Connect wallet to buy</code> . Amount and settings survive the Privy round trip
First run, no SOL	Three-step explainer replaces the panel body — but never over a block: it collapses to <code>We block buying on this coin — see why below</code> above a full-weight block strip, primary becomes ghost <code>Find a safer coin →</code> . Precedence: blocked, then unverified, then checks-pending, then no-route
Insufficient balance	Not disabled — cyan <code>Add 0.34 SOL</code> , opening the funding sheet with the shortfall prefilled
Back from funding, price moved	<code>When you left, \$47.30 bought 231,402 FERRY. It now buys 96,110 (-58%).</code> at equal weight, <code>Buy anyway</code> / <code>Change amount</code> . Explicit tap past 5% drift; intents older than 30 minutes or for another mint are discarded
Trade failed, transaction exists	Slippage revert or on-chain failure: <code>Price moved more than your 1% limit. Nothing was spent except the network fee. with the signature and a Solscan link</code> , plus <code>Retry at 3%</code> (pre-computed) and <code>Retry same</code> . Expired blockhash reads <code>did not land in time</code> ; retry bumps priority one tier, and from Turbo it retries at Turbo and says so
Trade failed, no signature	<code>We lost contact before confirming. Check your balance before retrying.</code> + <code>Check status</code> . Never auto-retry — that is how a user gets filled twice. With a signature but no status: <code>Still confirming (0:47)</code> , polling 90s
Declined in wallet	<code>You cancelled. Nothing was sent.</code> in --muted, not red. Declining is a normal outcome, not an error
Filled	<code>Filled — 231,402 \$FERRY</code> , sub <code>1.00 SOL · avg \$0.000204 · you were 41 minutes early</code> , signature copyable; the position card flashes in behind
Match revoked while held	Badge greys: <code>We are no longer confident this coin came from Last Horn.</code> The claim is withdrawn out loud, not left as a stale boast

STATE	WHAT THE USER SEES
Cost basis stale	Chain qty differs by >0.5%: PnL renders – with You have traded this outside Insidor – we cannot compute your cost basis.
Two wallets hold it	Position card shows the active wallet only, with a 2 wallets hold this chip. The frozen earliness belongs to the wallet-scoped row
Token dead	This coin appears to be dead. Liquidity \$0. Buy closed, sell open

What it needs

Reused: `narratives` , `narrative_posts` , the trades/holders/top-traders endpoints, the DexScreener embed logic, the format helpers.

New, none of which exists today. `narratives.promoted_at` (write-once, NULL for all pre-existing rows). `story_clocks` (narrative_id, promoted_at, t_mint, t_ct, t_crypto, lead_time_min, earliness_eligible, unmeasurable, gap_window). `coin_matches` (narrative_id, mint, relation, verdict, score, s_img/s_text/s_tick/s_mint_in_post, mint_time, delta_minutes). `coin_mints` (mint, minted_at, creator, program, image_url, phash) and `mint_stream_gaps` . `crypto_mentions` . `tokens` as a mint-keyed table (mint, ticker, name, decimals, token_program, logo_url, pair_address, socials) **plus** `liquidity_usd` and `liquidity_updated_at` , without which the sibling sort does not compile — minutes-stale is fine for a tiebreak. `token_safety` (authorities and source, transfer_fee_bps, has_transfer_hook, sell_route_ok, top10_pct). `holdings` unique on **(user_id, wallet, mint)** — not (user_id, mint), or a second wallet corrupts both cost basis and the frozen earliness. `trades` with `request_id` unique and `sol_usd_at_trade` . `clock_health` (sensor, last_heartbeat).

Routes: `POST /api/order` , `POST /api/execute` , and `GET /api/txstatus?sig=` — without it, "a fill is confirmed by signature status, never inferred from a balance change" has nothing to call. Poll at 1s for 15s, then 3s.

Before holdings ship: Privy configured as a third-party JWT issuer on Supabase with `sub` mapped to `user_id` ; RLS owner-read on `holdings` and `trades` , public-read on `coin_matches` and `story_clocks` ; both of the latter added to the realtime publication; and a refresh loop calling `setAuth()` , because Privy tokens live about an hour and without it every user silently drops off the live feed.

Build order

1. Fix the safety endpoint — Helius as primary authority source, three-valued absent/null/present logic, the full RugCheck report for LP and holders, `token_safety` cache. Ships alone, and stops telling every user that every token is safe.
2. Fix the age lookup — missing `pairCreatedAt` fails closed to null; age becomes `min(pairCreatedAt)` across all Solana pairs until `coin_mints` lands.
3. Add `promoted_at` and `story_clocks` , with the NULL-forever policy in the migration and a never-clobber guard in the persist layer. Nothing about earliness renders honestly before this.
4. Migrate to Jupiter Swap V2: order, execute and txstatus routes, server-side referral injection, referral token accounts initialised for wSOL and USDC with a boot assertion.
5. Rebuild the trade panel on those endpoints — fee row and popover, real route/impact/priority values, new presets, Auto slippage under 30 minutes, SOL-price fallbacks.
6. Implement the buy machine end to end with `trades` behind it. Ship every failure state in the first cut; lost-contact and still-confirming are not polish, they are where trust is won.
7. Add `coin_matches` and the day-1 non-ML matching rules. This is what makes the origin band possible and it needs no models.
8. Build the band on the ranked-row grid, plus the evidence popover, the live-lead line, and the clock-health gate on `FIRST` .
9. Build the brand-new tiers and enforce – versus `$0` across the stat grid. This is the modal case, not an edge case.
10. Add the four buy-side blocks, sell-route check first, with the sell-side transfer-fee disclosure alongside it.
11. Add `holdings` , the position card, and the earliness snapshot-and-backfill job including the negative and pending cases.
12. Sell sheet, with the reverse-quoted `Take back what I put in` and the no-buyers state.
13. Add-money sheet: computed end-to-end arithmetic, the intent return contract, the first-run explainer and its precedence rules.
14. Realtime on `coin_matches` and `story_clocks` . An unsure match upgrading to confirmed in front of a user — band turning cyan, lead counting into place, Buy appearing where there was none — is something nobody else can render.

05. Creating coins

What it is

Creating is what the product does when a story has no coin yet — the other half of the same sentence as buying. The primary button is a pure function of how many coins the matcher stands behind, and at zero it says Create. The job: turn a detected story into a live pump.fun coin in two steps and one signature, with the source post locked at the top so a coin can never be made out of nothing, and with the naming step doing the thing the user can't — knowing which phrase 44 posts are already repeating.

How it looks

Entry. Not a page. A button that appears only where the coin count is zero: the story footer (`Create coin`), a small `Create` pill in the market-cap column of a stories-lane row in Live feed, `Create Coin` on a viral post row (which carries *that* post, not the story's top post), Search's no-coin-yet rows and empty state, and one quiet secondary on a coin page — `Make your own coin` — for the deliberate second coin. Every one reads a single server row and nothing else: `select can_create, buy_count, unsure_count from story_cta_v where id = $1` . No client-side coinability logic exists anywhere. (The pill sits in Live feed's stories lane today; if stories become their own tab it travels unchanged.)

Step 1 · Name it. A 472px sheet — the same width as every other sheet, because this must not read like a fintech onboarding flow to someone who arrived from a ranked table.

Locked at the top: the source post. 42px thumbnail, `@nordkyst_ferge` , two clamped lines of text, and above it a 9px mono label `STORY · CANNOT BE CHANGED` . Down its right edge, three mono cells:

`SPOTTED 31m ago · CRYPTO TWITTER not yet · STORY accelerating`

`CRYPTO TWITTER` flips to `6m ago` in a red `.timing-pill.timing-late` the moment `t_crypto` fires — same border, size and weight as the early state. No closed `+31m early` badge here: on the create path no coin exists by definition, so lead time is an interval still running, and a closed figure would state a final number for a live measurement. `STORY` reads `accelerating` / `cooling` / `peaked 11m ago` from the lifecycle field the snapshot grid already computes. The question a creator is asking is "is the window still open," and these three cells answer it. They are also the part a competitor can't rebuild without a detection pipeline.

Below: a 96px crop preview left; coin name (32 chars) over ticker (13 chars, `$` prefix, sanitised to `[A-Z0-9]` per keystroke) right. Under them `.namer-why` , the trust line — `named from the phrase repeated in 31 of 44 posts` , or `the cashtag in the post` , or `from the video's on-screen text` . Then three alternative chips chosen for diversity of kind, not next-best score — a compression, an entity, one from a different post — each with dim provenance beneath (`in the caption` / `said in 34 posts` / `from the replies`), and `🔄 Try another` with a `2 left` counter.

Then `.coll` , the availability line: `Enter a ticker for your coin.` → `✓ $CHILLGUY is free on Solana — you'd be first to deploy it.`

Then the fixed facts, as a mono key/value block with right-aligned values — not sentences:

SUPPLY	1,000,000,000 · FIXED
YOUR SUPPLY	0
FREEZE	NONE
TRANSFER FEE	0
LIQUIDITY	SELF-SEEDED

`YOUR SUPPLY 0` is first and deliberate. The most reliable first-timer misconception about a bonding curve is that creating a coin means receiving the coins. With the first buy defaulted to None — which it is — the modal first launch leaves the creator holding exactly zero, and if no screen says so they'll tap Trade on the success screen, see nothing, and conclude they were robbed.

Then the fee split panel (§07) and two disclaimers — `Not created or endorsed by @nordkyst_ferge.` and `Anyone can make a coin from this story, including someone else, right now.` — placed *above* the optional first-buy chips (`None | 0.1 | 0.5 | 1 SOL` , default None), so the load-bearing disclosure is above the fold and the optional field is what scrolls.

Footer: `COST TO LAUNCH ~0.02 SOL ≈ $4.02` left, `Review $CHILLGUY` right. Nothing signs from this screen. The cost label opens an itemised popover ending in `Insidor's cut 0.0000 SOL now — 10% of your fee stream (0.1% of each trade)` , because a breakdown that omits our cut reads as a claim that we charge nothing.

Editing the image expands **inline**, replacing the identity block — never a second modal, since opening one destroys the other. 1:1 crop frame, drag, zoom, Fit/Fill, and for video a scrubber of five frames at 10/25/40/55/70% of duration with the auto-pick outlined in cyan.

Step 2 · Confirm. Same sheet, body replaced. 72px image, **\$CHILLGUY** in Clash Display. A red-bordered, non-dismissible **PERMANENT** list: ticker and name can't be edited, image can't be replaced, the split is written once, the coin exists on Solana forever — Insider can hide it, can't delete it. Then mono rows: **Story** (with ↗), the same three earliness cells **re-read at mount**, **Signed by**, **Creator on-chain** (only when it differs from the signer), **First buy**, **Your \$CHILLGUY** (**0** or **about 4,120,000**), **Total cost**, **Your balance after**. Beneath, a pulsing cyan dot: **Re-checking nobody minted this...** → **✓ Still free, checked 2s ago**. Then the loss line in plain muted text, never a red box, because a box gets dismissed and text gets read: **Most coins go to zero. If this one does, you lose the 0.5 SOL you're putting in and earn no fees.**

Create \$CHILLGUY is the only button in the flow that signs. On press it reads **Waiting for your wallet...**, and backdrop-click and Esc are dead until resolution.

Signing replaces the body with four mono rows — upload, sign, land, assign the split. Success is declared on step 3; step 4 failing gives a retryable amber band, never a failure screen.

Success gives the lime check, **\$CHILLGUY is live**, the line people screenshot (**Just a Chill Guy · made from "Norwegian ferry horn" · 4 minutes after we spotted it**), the full mint with **Copy CA**, Solscan / pump.fun / DexScreener links, a share card, **Trade \$CHILLGUY**, and **Done** — which returns to the **story page**, so the user sees their coin in the story's coin list. That's the proof it attached.

How it works

The namer never authors a token. A memecoin's name is the phrase people are already repeating — WIF keeps the misspelling because the misspelling is the meme; MOODENG is a name; HAWKTUAH is an utterance. So this is retrieval, not generation, and retrieval is ours to do because we hold the whole cluster and pump.fun holds one form.

Stage 0 runs at PROMOTE, no model, ~4ms, cached. Six sources with weights: **cashtag** 1.00 (**\\$([A-Za-z][A-Za-z0-9]{1,12})\b** over post text *and* sample replies across the cluster), **onscreen** 0.85 (OCR — on TikTok the caption is the meme more often than the audio), **ngram** 0.80, **entity** 0.75, **llm_prior** 0.35. A base58 mint pasted in a post isn't a name candidate; it's a duplicate signal.

```
distinctiveness(span) = (df_in_story/n_posts) × ln( N_posts_24h / (1 + df_global_24h) )

score(c) = 0.34·w_source + 0.26·min(df_in_story / max(3, 0.25·n_posts), 1)
          + 0.18·clamp01(distinctiveness/10) + 0.12·len_fit + 0.10·(1 - collision_pressure)

len_fit          = 1.0 (4-9ch) | 0.7 (3, 10-11) | 0.35 (12-13) | 0
collision_pressure = clamp01( log10(1 + max_liquidity_usd of live tokens with that symbol) / 7 )
```

llm_prior can never be primary without corroboration from one of the other four. Normalisation uppercases and joins with no separator; over-length drops stopwords first (**JUSTACHILLGUY** → **CHILLGUY**), then the most distinctive word, then initials — **never truncate mid-word**, **CHILL6** signals low effort to every trader who sees it. Deliberate misspellings are preserved. **collision_pressure** exists because a symbol already carrying a \$10M token makes your coin unfindable — a distribution problem, not a legal one.

One **claude-haiku-4-5** call fires on form open and caches on the story for 20 minutes. It ranks and title-cases grounded spans; it does not invent. Then the gate, server-side — any failure discards the *entire* model output and the deterministic ranking stands: (1) **source_span** must be a case-insensitive substring of the concatenated post text, OCR, replies and entity keys; (2) **^[A-Z][A-Z0-9]{2,12}\$**; (3) lengths; (4) not blocklisted; (5) **symbol = norm(source_span)** or a whole-word subset of it — blocks creative leaps, allows stopword-dropping; (6) the generic test. A ticker appearing nowhere in the corpus cannot survive rule 1.

The same gate runs on any prefilled symbol. A ticker arriving from a Search row is not privileged because a user clicked it: rules 1, 2, 4 and 6 run before it touches the field, and we only prefill from **narrative_tickers.source IN ('cashtag','mint_in_post')**, never **'llm'**. An ungrounded suggestion drops to alternative slot 3 labelled **suggested, not found in the posts**. Otherwise we'd launder model output as user intent through the front door of the gate we just built.

The form never waits on the model. Haiku's p50 is ~700–900ms; the sheet opens in under 100ms on the deterministic #1 with **.namer-why** shimmering. The model result swaps in only if neither field has been touched; if the user typed anything it silently becomes alternative slot 1. The namer is an accelerant, never a gate — even **namable=false** opens the form.

The image is the post's media, cropped and re-hosted. Never generated. It's the evidence, and a generated logo severs the visual link that makes the coin recognisable to the next 10,000 people who watch the same video — recognition is distribution. It's also what our own matcher needs: a crop of the post media confirms at **S_img = 1.00** on pHash

distance ≤ 6 , while a generated logo lands in the SigLIP channel capped at 0.60, **which cannot confirm alone** — we'd be minting coins our own pipeline can't verify. Text-only posts render the post itself in Clash Display on `#0C0C0E` : not art, a screenshot of the evidence, still pHash-matchable.

The anti-duplicate check is Search's reason to exist, run inside the create flow — on open, on every 250ms-debounced keystroke ≥ 2 chars, and again before signing, because ~ 40 seconds elapse between open and sign and that's exactly the window in which someone else mints.

Per-keystroke it hits local sources only: `coin_index` by `symbol_norm` plus an explicit `similarity(symbol_norm, $1)  $\geq$  0.6` (the bare `%` operator tests a GUC defaulting to 0.3 and would return double the candidate set), and the in-memory PumpPortal create-event ring buffer for the last 15 minutes. Ordering is `liquidity_usd desc`, `first_mint_at asc` — never 24h volume, which is what selects a \$1.8B major for a fresh story. Jupiter fires exactly twice per session, server-side, behind a 30s per-symbol cache shared across users, so a hot symbol costs one call for everyone; keyless it measures 0.5 RPS and a 250ms debounce would 429 on the first user.

The **pre-sign re-check reads the mint stream first**, not the index and not Jupiter. The coins it exists to catch are 0–90s old, and at that age there's no DexScreener pair and Jupiter returns null audit data — both fallbacks are blind to exactly the window that matters. Stream liveness, not index freshness, is the health metric.

BAND	FIRES WHEN	WHAT HAPPENS
D-CONFIRMED	matcher confirmed a coin for <i>this</i> story	Form doesn't open — entry reads <code>Buy \$TICKER</code> . Landing mid-session swaps to the interstitial.
D-STRONG	exact symbol, minted within 10 min of the earliest post, $\geq$ \$1k liquidity, image-or-name agreement	Card expands, <code>.coll</code> amber, cost line gains <code>You'd be the second \$FERRY. The first has \$18.2K in it.</code> Bordered secondary <code>See \$FERRY</code> . Primary stays <code>Review \$TICKER</code> .
D-WEAK	exact symbol but >48h older, zero liquidity, or a different subject	One amber line: <code>\$PUMP already exists – a \$1.8B token from 2024, unrelated to this story. Yours would be a different coin with the same ticker.</code> Primary unchanged.
D-IMAGE	pHash ≤ 6 against a recent mint, symbol differs	<code>This image is already a coin – \$CUPS, made 12m ago, \$4.1K in it.</code> Secondary <code>See \$CUPS</code> . Primary unchanged — same image, different ticker is a fork.

D-STRONG does not flip the primary to Buy, and that's load-bearing. This band only fires on coins the matcher did *not* confirm — if it had, the form wouldn't have opened. Flipping there ships a second, looser, undocumented matcher and hands it the Buy button the real one withheld. It's also trivially attackable and pays the attacker: mint \$FERRY with the post's image inside the window, seed \$1k, and we'd point every would-be creator at your coin. Instead the D-STRONG feature vector goes to the real matcher; if it returns CONFIRMED, D-CONFIRMED handles it. One matcher, one threshold, one Buy button. UNSURE behaves the same, plus one more reason: score-gated suppression is a denial-of-creation vector — mint a decoy and you suppress everyone's create flow on a hot story.

D-IMAGE needs more than distance. Against a corpus where 58.6% of new mints carry byte-identical logos, `hamming  $\leq$  6` fires constantly, hardest on the low-information images where it means least. So: suppress when the matched SHA-256 group exceeds 3 or the pHash group exceeds 5, require `liquidity_usd > 0` on the match, tighten to `$\leq 4$` for non-singleton SHA groups. `bit_count()` isn't indexable — this precomputes into a BK-tree or it doesn't ship.

The transaction. The mint keypair is generated and persisted encrypted **before** the first signature, so a retry reuses it and a late confirmation fails with `account already in use` rather than minting twice. `/api/launch/prepare` returns

a transaction already partially signed by the mint keypair — "one signature" is true from the user's side, and the server contract must say so. The first buy is bundled into the launch tx; unbundled, the sniper bots win the gap.

FLAG: exact `create_v2` rent is unmeasured, so the UI shows `about 0.02 SOL` , never a precise figure, and the balance gate requires 0.03 so we never quote a number the transaction exceeds. `is_mayhem_mode` and `is_cashback_enabled` are required instruction fields and undecided product parameters — they change what the safety block means. Every `@pump-fun/pump-sdk` fee-sharing symbol is a placeholder until the SDK is installed and its types grepped; the README documents functions absent from the shipped types.

The first buy executes on the bonding curve inside the launch tx, so the Jupiter integrator fee doesn't touch it — but every later trade routes through the buy path, where that fee has a 50 bps floor, making 0.5% per trade the cheapest possible user cost.

Propagation. On landing, one transaction writes `launch` , then `narrative_tickers` with `source='insidor_launch'` , `match_state='CONFIRMED'` , `match_score=1.0` — **bypassing the matcher, because we have ground truth: we minted it from this story** — then `coin_index` with `is_insidor_launch=true` so the next user's duplicate check catches it in seconds, then two follow rows, the coin *and* the story, because following a story must never silently become following a coin. Realtime flips every viewer's primary to `Buy $CHILLGUY` in under a second. The new coins-lane row carries a `NEW` badge with **all nine numeric cells as — , never 0** .

The count behind every button. `buy_count = count(match_state IN ('CONFIRMED','ADOPTED'))` — an ADOPTED coin is a story that legitimately attached to a pre-existing token, and it must not render Create over a real buy; it gets a `.timing-pill` reading `existed before this story` rather than an earliness badge. `can_create = coinability_tier = 2` . The classifier's `coinable` flag is a naming-quality judgement and **does not** gate the CTA — a story with no obvious ticker candidate is precisely what the harvester exists to fix. The 50–70% figure in the pipeline research is that suppression rate, not a create rate.

Rate limits. `launches_last_1h` / `launches_last_24h` read into `story_cta_v` . Above 3/hour the CTA becomes a cooldown chip rather than vanishing. A per-poster daily cap too, because 40 coins off one creator's posts is what produces a takedown — and uncapped, spam is directly monetised for us.

States

STATE	WHAT THE USER SEES
Loading	Sheet open <100ms on the deterministic candidate, <code>.namer-why</code> shimmering
Namer unavailable / >2.5s	Deterministic list stands: <code>Suggestions are off right now – pick a name yourself.</code>
<code>namable = false</code>	Fields empty, <code>We couldn't find a name in this story – no phrase repeats across the posts. Pick one yourself.</code> plus five <code>words from the posts</code> chips. Create still reachable.
Ticker free	<code>✓ \$CHILLGUY is free on Solana – you'd be first to deploy it.</code>
Over 13 chars	<code>Ticker is 13 characters max. We'd have to cut it to \$CHILLGUYWORL.</code> + <code>Use that</code> . Never silent truncation.
Blocked / leading digit	<code>We don't allow this ticker.</code> (no list, no explanation) · <code>Tickers start with a letter.</code>
Unverified match	Muted card above the grid: <code>Possibly already made: \$KANG – we're not confident it's from this story.</code> Buy is not rendered at all; primary stays Create.
One coin lands mid-form	<code>\$FERRY was made 40 seconds ago.</code> → primary <code>Buy \$FERRY</code> , then <code>Make a different one</code> , <code>Show me the story</code>
Two coins land mid-form	<code>Two \$FERRY coins were made while you were here.</code> Both cards stacked with liquidity, holders, age. Primary <code>See the 2 coins</code> . Never a Buy aimed at whichever row sorted first.
Duplicate <90s old	Money row dropped: <code>made 12 seconds after the post · nobody has traded it yet</code> , from the mint stream. Never <code>\$– in it · – owners</code> .
Brand-new token after launch	Every numeric <code>–</code> , chart <code>No live SOL pair for this token yet</code> , age <code>just now</code> , DexScreener disabled with <code>chart appears once there's a pair</code>
Negative lead time	<code>CRYPTO TWITTER 6m ago</code> in red <code>.timing-pill.timing-late</code> , identical weight to the early state; subtitle <code>crypto Twitter got here 6 minutes before us</code> . Same on the share card and the public <code>/c/{mint}</code> route.
<code>promoted_at</code> backfilled	Earliness sentence and share-card line suppressed — falls back to <code>made from "Norwegian ferry horn"</code> , no minute figure. A fabricated screenshot number is the most expensive dishonesty we can ship.
Story merged since promote	Lead time may be inherited from the merge target — rendered only when no merge has occurred since <code>promoted_at</code> , otherwise marked.
Mint stream down >60s	<code>We can't see brand-new coins right now – someone may have minted this in the last 4 minutes.</code> Non-blocking.
Jupiter bucket empty · index stale	<code>Live check unavailable – checked against our index only.</code> · <code>Checking against coins up to 4 minutes old.</code>
Pipeline stale	<code>Story data last updated 14 minutes ago.</code> No block — creation doesn't need a live pipeline.
Wallet not connected	Form fully usable; step 2 reads <code>Connect wallet to create</code> → Privy → returns with every field preserved
Insufficient SOL	<code>Add money to create</code> + <code>You need about 0.03 SOL. You have 0.004 SOL.</code>
Rate limited	<code>You've launched 3 coins in the last hour. The next one unlocks in 14 minutes.</code>
Tier 1	No CTA anywhere. Via stale link: <code>This story isn't available to coin.</code> + <code>Back to the story</code> . No mechanics, no <code>why?</code> — explaining the boundary is a bypass manual.
Tier 0	Story never surfaces. Not a create-flow state.
Image unfetchable	Rendered-text card + <code>We couldn't fetch the video's image. Using the post text instead – or upload your own.</code>
Wallet rejected	Returns to step 2 unchanged, one muted line: <code>Cancelled. Nothing was spent.</code> No red, no icon, no "are you sure?"
Unconfirmed at 45s	<code>We haven't seen this land yet. It may still go through – we're watching for 60 more seconds.</code> Never claim failure while a signature is unresolved.
Blockhash expired	<code>That transaction expired before it landed. Nothing was spent. Try again – the details are still here.</code> Retry reuses the same keypair.
Any pre-signature failure	<code>Nothing has been signed and nothing has been spent.</code> Verbatim, every time.
Split pending / failed at 24h	<code>Until it lands, 100% of fees go to you.</code> → <code>We couldn't write the fee split. Nothing is lost – 100% of fees go to you. We've flagged this.</code>
Tapping a duplicate's coin page	Opens in a new browser tab , as the post permalink does. One modal root; in-app navigation destroys the form.

What it needs

Used as-is: `narratives.{title, blurb, platforms, combined_views, lifecycle, top_post_id}`; `narrative_posts.{handle, text, media_url, media_type, views, sample_replies}` — replies are a first-class namer input, because replies are where a phrase proves it's being repeated; `narrative_tickers.{ticker, mint_ca, liquidity, mcap, holders, first_deployed}`; `post_meme_scores.suggested_ticker`, whose **only** sanctioned consumer in the product is this form, ranked last at 0.35 and never primary without corroboration.

Does not exist yet. On `narratives : promoted_at` (set once, immutable) plus `promoted_at_backfilled boolean` so the earliness claim can be suppressed on backfilled rows; `coinability_tier smallint`; `subject`, `subject_type`, `entity_keys text[]`; `titled_at`. On `narrative_tickers : source`, `match_state`, `match_score`, `match_features`. On `narrative_posts : ocr_text`, `media_sha256`, `media_phash bit(64)`, `media_local_url`. New tables: `launch`, `namer_run`, `fee_share`, `coin_index`, plus `app_user`, `poster`, `follow`. New health field: `pipeline_health.mint_stream_connected_at`.

`namer_run` is written **at form open, not at submit**, with `outcome` in `accepted | edited | abandoned | typed_over` and `time_to_first_keystroke_ms`. Abandonment is the strongest negative signal the namer produces, and a schema that only records submissions fits the weights on a hits-only sample.

Non-DB gaps, all real: `coin_index` has no producer today; there is no OCR service, no ffmpeg frame extractor, no rendered-text image generator, no share-card renderer, no `/c/{mint}` route, and no `/api/launch/*` endpoints — today's `deployFiled()` fabricates a base58 string and mutates a module-scope array. Unhandled anywhere: `pump.fun` controls `toggle_create_v2`, so a third party can switch this whole feature off.

Build order

- 1. **Migration 001** — every column the rest reads. Ships alone, changes no UI.
- 2. `story_cta_v` + **server-side gating** on all existing Create affordances. Tier ≤ 1 renders nothing, enforced in the query. The safety floor, before any new create UI exists.
- 3. **Deterministic harvester** at `GET /api/story/:id/name-candidates`, replacing `deriveTicker()` in the current sheet. Better naming, zero model spend, zero new UI.
- 4. **Image pipeline** — fetch, SHA-256, re-host, crop, frame selection, rendered-text fallback. Also kills the placeholder-image hack across the app.
- 5. **Haiku namer** + `namer_run` + the gate + the never-wait render contract. Feature-flagged; the deterministic path is the permanent fallback, not a stub.
- 6. **Anti-dupe, read-only**. 6a Jupiter symbol search with the shared cache — three of the four bands on its own. 6b the create-event stream into `coin_index` and the ring buffer. 6c pHash + BK-tree for D-IMAGE.
- 7. **Step 1, wallet-optional, ending at Review**. No signing, no chain. Shippable internally as a naming tool.
- 8. **Real launch** — IPFS with Pinata fallback, `createV2Instruction`, bundled first buy, partial-sign contract, Privy raw bytes, send and confirm, keypair retry safety, the full failure taxonomy and its copy.
- 9. **Propagation** — the single-transaction write, both follow rows, Realtime, optimistic rows with `-` not `0`.
- 10. **Success + `/c/{mint}` + share card**, rendering negative lead time at the same weight as positive, and an X compose intent that never auto-posts.
- 11. **Fee split**, gated on D2, shipped in one release with the disclaimer.
- 12. **Rate limits, regeneration UX, and the label loop** refitting the ranking weights on `outcome` — the only step that makes the namer improve rather than merely exist.

06. Search and Watchlist

What it is

One box that takes a contract address, a `$TICKER` , an `@handle` or plain English and answers one question: does a coin already exist for this, and if so, which one is real. The star (★) is the other half — it turns "I'm waiting on this" into a notification the moment a watched story actually mints. Search is the anti-duplicate surface; the star is what you press when search tells you nothing exists yet.

How it looks

Tab two of three. Near-black page, one 56px input across the column, cyan focus ring, and inside the field on the right a small mono chip reading `CONTRACT` , `TICKER` , `POSTER` , or nothing. That chip is the entire mode UI — the user never picks a mode, they watch which one fired.

Under it, a chip rail with live counts: **All • No coin yet (34) • Crypto hasn't seen it (11) • Just made (12) • Trending now (28) • Finished (214)**, scoped to the query or global when the box is empty. `No coin yet` is the default view: every promoted story nobody has minted, sorted by how fast it's spreading, Create one tap away. Below, a mono strip — `34 results • 88ms • updated 4s ago` — plus one disclosure, `results with a tradeable coin rank slightly higher` . When the pipeline is behind, an amber band: *"Pipeline stale — newest post is 47m old."*

Results are **sectioned, not interleaved** — three object types, three column grammars, and a terminal user scans columns. The winner is lifted into a normal-height row with a 2px cyan left edge, a reason (`exact ticker match`) and a right-aligned button; suppressed when the top two scores land within 8% of each other, and suppressed entirely when the winner is a coin with no story, because "exact ticker match" is not an answer to "is this the real one."

Stories (≤6): rank · 56px thumb · title with `<mark>` hits over a one-line snippet · views with `1.2K/min` beneath · age since `promoted_at` · Lead (`+31m early` lime / `-6m late` red / `live`) · action. Rows target 54–60px; twelve results above the fold at 900px is an acceptance criterion, not a preference. Lead is sortable everywhere it appears.

Coins (≤8): `# • Token • Price • 24h • Mcap • Liq • Age • MATCH • Story • action` . The Story column is the whole differentiator against DexScreener and it is an independent link. MATCH is categorical — `CONFIRMED` lime, `UNVERIFIED` amber, `ADOPTED` cyan, `NO STORY` grey — and clicking it opens the evidence: the post's image beside the token logo, `minted 4m after the post` , and which channels fired as words (`image + name`). The numeric score is stored for calibration and rendered nowhere.

Posts (≤10, folded to 3): platform badge · two lines of post text · `@handle` + followers · views · age · `↗ post` `→ story` . Never a Create button on a post row — creation is always anchored to a story.

A pasted address drops the sections: a compact identity card (52px logo, `$SYM` , truncated mint with copy, price and 24h) over a strip of 5m · 1h · 24h · Mcap · Liquidity · 24h vol · Holders. Under the ticker, the MATCH stamp and one plain clause — *"matched · image + name · minted 4m after the post."* Then the origin story, then **Posts containing this address** (your "search a CA, the post appears," which mechanically is the strongest evidence channel the matcher has), then a collapsed same-ticker family when two or more coins share the symbol. At the bottom one dim line: `resolved from our index • 4ms` or `resolved live from DexScreener • 240ms` . A terminal user needs to know whether they're reading indexed truth or a cold single-source lookup.

`$FERRY` is the anti-duplicate screen, and it is a sentence — a full-width band, Clash 20px, one of six:

SITUATION	VERDICT
No live mint	<code>\$FERRY is free on Solana — you'd be first.</code> (lime)
1 mint, matched	<code>\$FERRY exists. 1 coin, matched to a story.</code> (cyan)
N mints, 1 matched	<code>\$FERRY exists 4 times. 1 is matched to a story.</code>
N mints, none matched	<code>\$FERRY exists 4 times. None matched to a story.</code>
N mints, 2+ matched to different stories	<code>\$FERRY exists 4 times. 2 are matched to different stories — we can't tell you which one you mean.</code>
Lookup failed	<code>We couldn't check whether \$FERRY exists right now.</code> — Create suppressed

Beneath it the collision line, in words and never a number: *"PUMP is a common ticker — 4 live coins share it."* Then the family table — `# · Mint · Name · Age · Mcap · Liq · Vol 24h · MATCH · action` — **sorted by mint time ascending**, earliest row badged `FIRST` , with a caption saying why: *"oldest first — the first mint is usually the real one."* Every competitor sorts by volume, which ranks whichever copycat got pumped. Buy appears only on rows that are confirmed *and* currently tradeable, and nowhere at all in the fifth case. Below: stories proposing `$FERRY` with no coin, each tagged `cashtag` (someone typed it) or `suggested` (an LLM proposed it). Then Create — full-width primary when the ticker is free, demoted to a dim link *"Make another \$FERRY anyway →"* when a coin exists. Demotion, not removal.

Zero results is the best state in the feature: *"Nothing called 'ferry horn' yet. No story, no post, no coin. That is usually the good news."* + `Make the first $FERRYHORN coin` , prefilled.

The star sits inline in every row and as `☆ Watch` / `★ Watching` on story and coin pages. No wallet, no account — saved against a device id, claimed on first login. It never silently converts: when a watched story mints, the follow stays a *story* follow (a story can mint again) and an explicit `+ Watch $CUPS too` writes the second row. **/you** stacks by urgency, empty sections unrendered: **Just minted** (confirmed in 24h, unopened, lime-pinned) · **Waiting — no coin yet** · **With coins** · **Coins you watch** · **Gone quiet** (collapsed, `Clear all 7`). Watchlist rot is the real reason watchlists get abandoned. The empty state teaches — *"Tap ★ on any story and we'll tell you the moment somebody mints it. No wallet needed."* — then three **real live rows** starrable in place, and one honest stat: *"Last week, 61% of watched stories got a coin within 3 hours. 34% never did."*

The notification is a 380px toast, lime edge. Line one is the story then the ticker — *"Wrong Name is now \$CUPS"* — because the user followed a story and must recognise it before parsing a ticker. Line two: `Minted 2m ago · $18.4K · you're 31m early` ; when the lead is negative, same slot, same size: `crypto traders got here 6m before us` . Two buttons, and above the primary — always, at the weight of the lead line — `fee 0.50% · impact 2.3% · you get ≈1.42M $CUPS` . If the toast can't render a live quote it navigates instead of trading; 400ms is not worth an undisclosed fee on a push-driven buy. Miss it and the next open shows a **while-you-were-away** band at position one, which is also the only place push permission is ever asked.

How it works

`parseQuery` is pure and ordered: 32–44 base58 chars decoding to exactly 32 bytes → CONTRACT; `$XXX` → TICKER; a bare 2–13 alphanumeric word → **ambiguous, run ticker and text and section both**; `@handle` → POSTER; else TEXT. Ambiguity is resolved by showing both, never by guessing. A pasted CA skips the 250ms debounce — a pasted address is never exploratory.

Three retrievers fused by Reciprocal Rank Fusion, because `ts_rank_cd` , trigram similarity and cosine distance sit on incomparable scales and normalising them needs a calibration set we don't have. FTS carries relevance; trigram at 0.30 (0.18 on ticker columns, since ticker drift is one or two characters) carries FERRY/FERRIE/FERY; pgvector carries paraphrase. Each returns its own top 100; `RRF = Σ w/(60+rank)` , weights 1.00 / 0.60 / 0.80. Eligibility is hoisted into **one eligible_narratives** CTE — `promoted_at IS NOT NULL , status IN ('open','dormant') , coinability_tier <> 'never'` — that all three legs join, posts and coins inheriting from their narrative. A tier- `never` story must not be reachable through the vector leg because somebody paraphrased its title; a CI test seeds one and asserts zero rows.

`score = (0.70·rel + 0.30·vir) × boost × typePrior` , where `rel = RRF/max(RRF)` and `vir = ln(1+heat)/ln(101)` . There is no separate recency term — heat is already age-decayed and p95-normalised, and a third decay penalised a 90-minute story three times over. What replaces it is the thing that *is* the product: ×1.20 when `t_crypto IS NULL` (crypto hasn't touched this), ×1.10 when `lead_time_min > 20` . Also ×1.35 exact ticker equality, ×1.15 query-in-title, ×1.05 for ≥1 confirmed coin (disclosed on screen, measured in `search_log`), ×0.60 when `needs_review` . Relevance dominates because a search box that reorders your query by popularity is a trending board wearing a search box. Earliness is a boost, a sortable column *and* a chip — otherwise this is a generic screener any company with no detection pipeline could ship.

Snippets come from `ts_headline` but **not as HTML** — Postgres doesn't escape the source document and that source is scraped social text. Delimiters are sentinel control characters; the fragment is escaped client-side, then split and rendered as real `<mark>` elements.

The CA ladder is four rungs, each a write-through cache: `narrative_tickers.mint` (~4ms) → `coin_index.mint` (~4ms) → live DexScreener (~250ms) → RPC owner check (~120ms; a non-token owner renders *"That's a wallet, not a token."*). `mint_time` is `min(pairCreatedAt)` across **all** pairs, never the best pair; missing means `null` , never `0` . In parallel with every rung: `posts WHERE text ILIKE '%mint%'` . The lookup layer returns a discriminated `status: 'found' | 'absent' | 'unavailable'` , never a boolean — today a DexScreener 429 and a genuinely free ticker are the same value, which means under load, on the hottest ticker, this screen renders "you'd be first to deploy it" and a full-width Create button. A zero-liquidity mint is a coin that exists — exactly the case this screen is for — and returns `found` with `hasMarket: false` .

One resolver, eight surfaces. `resolveAction()` takes two inputs, not one: confirmed identity **and** current tradeability. 0 confirmed → **Create** (or a non-interactive `Not mintable` pill at tier `no_create`). 1 confirmed and tradeable → **Buy \$FERRY**. 1 confirmed, not tradeable → **View** + `liquidity gone` or the red `Risky` pill. 2+ → **See the 3 coins**. Unsure only → primary stays **Create**, candidates show `UNVERIFIED` and `View anyway` , and Buy is not rendered at all. `match_status = 'confirmed'` is an identity assertion, not a tradeability assertion, and it never expires — a rugged coin would otherwise render Buy forever. Tradeable means liquidity above zero, no hard safety gate (mint or freeze authority, top-10 over 60%, live under \$3k liquidity, zero sells against 30+ buys) and price under five minutes old. Every market number carries a freshness treatment; a 40-minute-old mcap must not render identically to a two-second-old one.

Notifications fire on `confirmed_coin_count: 0 → ≥1` and only on `confirmed` . Unsure never notifies — pushing "your story minted" on a 0.6-confidence match is what the abstain band exists to prevent. An unsure that later promotes fires in-app only, with the honest lead ("*we confirmed this 40m after it minted*").

```
dedupe_key = coalesce(user_id::text,'dev:'||device_id) || ':' || narrative_id
              || ':' || floor(extract(epoch from now())/21600)::text    -- NOT NULL

topic      = coalesce('user:'||user_id, 'device:'||device_id)
```

Anonymous device follows are the highest-volume follower class and get the same delivery path. Coins confirmed within 90 seconds coalesce into one message — swarms mint three to eight in under a minute — and 2+ makes the copy "*Wrong Name got 3 coins in 90 seconds*" with `See the 3 coins` . A mounted toast **subscribes** to its narrative's confirmed count rather than snapshotting it; a second coin landing past the window re-resolves the button with an amber flash and "*another coin just appeared.*" Budget, mint to glass: p50 under 20s, p90 under 60s — a product requirement, not an SLO. Two alerts serve the thesis better and ship alongside: **window closing** (`t_ct` fires while confirmed count is still 0 — crypto found your story and there's still no coin) and **accelerating** (heat rank crosses into the top N, one per story). The first is the most on-thesis message we can send, and it drives Create.

The honest record (`Finished`) states its denominator and defaults to All as a route constant, not a remembered preference: `Stories 214 · got a coin 71% · never did 29%` , then `Of the 152 that got a coin: early 61% (median +23m), late 34% (median -7m)` , then a permanent `Abstained 22%` cell, amber only above 30%. Columns: `# · thumb · Story · We saw it · Crypto saw it · Lead · Coin · Outcome · Proof` , with absolute clock times, Lead sortable ascending in one click, and Proof as two external permalinks that belong to somebody else. No `Peak mcap` column and no `FLAT` outcome until `coin_snapshots` exists — a last-observed value labelled "peak" is a fabrication.

States

STATE	WHAT THE USER SEES
Empty box	The <code>No coin yet</code> list + six Try chips seeded from live data (2 hot tickers, 2 entity words, 1 recent CA, 1 handle) — never hardcoded
First-ever visit	<i>"Search what you saw on TikTok"</i> above the input, and a dismissible strip naming the four non-obvious columns: Ticker, Velocity, Lead, Match
Zero results	<i>"Nothing called 'ferry horn' yet... That is usually the good news."</i> + <code>Make the first \$FERRYHORN coin</code>
Story with no coin	The normal case. <code>Create coin</code> primary; ticker cell shows the proposed cashtag or <code>no ticker</code>
Unverified match	Amber <code>UNVERIFIED</code> , candidate shown, <code>View anyway</code> , Buy not rendered, primary stays <code>Create</code>
Confirmed but not tradeable	<code>View</code> + <code>liquidity gone</code> , or the red <code>Risky</code> pill. Never Buy
Two mints confirmed to two stories	Fifth verdict sentence; every family row gets <code>View</code> ; no Buy on the screen
Brand-new token, no market	Price / 24h / Mcap / Liq all <code>–</code> in grey, never <code>0</code> , never a green chip; <code>no market yet</code> under the ticker. Buy stays enabled — a bonding-curve buy is valid
<code>mint_time</code> null	<code>Age: unknown</code> , MATCH forced to <code>NO STORY</code> , Buy suppressed. Fails closed
Lookup unavailable (429 / timeout)	<i>"We couldn't check whether \$FERRY exists right now."</i> Create suppressed + retry. Never "you'd be first"
Valid address, nothing anywhere	<i>"Valid Solana address. No pair, no story, no post. Nothing has traded."</i> A result with the address echoed, not an error toast
Address is a wallet	<i>"That's a wallet, not a token."</i> + <code>View trader →</code>
Negative lead time	<code>-6m late</code> in red, same slot and weight as <code>+31m early</code> ; on Finished, a grey line: <i>"Crypto traders got here 6 minutes before us."</i>
Tier <code>never</code>	Absent entirely — no row, no count, no gap, no trace
Tier <code>no_create</code>	Row renders; action is a non-focusable <code>Not mintable</code> pill with <i>"Public tragedy — discussable, not mintable."</i> The story page still opens
Story aged out	One grey row: <i>"This story closed 12 days ago — see it in Finished →"</i> , not an empty page
Stale pipeline	Amber banner, results still render. If nothing passes the freshness gate, fall back to most-recent and say so
Degraded vector leg	Many narratives have no embedded post, so the meta strip reads <code>semantic recall partial</code> rather than implying full coverage
Search error	<i>"Search is down. The board still works."</i> + retry. Chips stay usable — plain table queries that bypass the fused function
Wallet not connected	Everything renders. ★ works via device id; Create opens connect inside the flow. Only the signature gates
Watchlist empty	The teaching state: copy, three real starrable rows, the honest 61% / 34% stat
Anonymous follower	Fully functional; a dim <code>saved to this device</code> once, on the first follow ever; delivery on the device topic
Story merged	Follow repoints to the survivor with a <code>merged into <title></code> caption for 24h — the one case where the target changes, and it is disclosed
Held for quiet hours	Every number recomputed at render: <i>"Minted 8h ago · \$4.1K now (was \$18.4K) · we told you 31m before crypto, 8h ago."</i> Never a replayed payload
Match retracted	In-app correction — <i>"we were wrong about \$CUPS — it is not from this story"</i> — in the away band, original notification marked corrected

What it needs

On `narrative_tickers`: `mint` — but **verify first** whether the live catalog already has `mint_ca`. `docs/schema.md` lists it and was generated from a live probe; the claim that it's absent is unverified, and if it exists we rename rather than adding a second column for one concept. Then `source` (`cashtag|llm|mint_in_post`), `relation`, `match_status`, `match_score` (stored, never rendered), `match_channels` jsonb, `mint_time` nullable. Without these, every Match stamp is a lie and the primary button can't be computed — today it keys off `ticker && mcap ≠ null`, the string matching that produced the live `$KANG` and `$PUMP` bugs.

On `narratives`: `promoted_at` (set once, never overwritten — the clock every lead-time claim is measured from; `created_at` moves on merge), `heat` / `heat_rank`, `coinability_tier`, `needs_review` / `coherence`, `search_doc` + GIN, `search_vec` + HNSW. Verify the embedding dimension **and model** against `post_embeddings` before writing the vector leg.

New tables: `coin_index` (search can't answer "how many \$FERRY coins exist" without it; holders belong here as a refreshed column with a `captured_at`, not one Helius call per visible row) · `app_user` · `follow` (+ `device_id`, `notify_mint`, `muted_until`, `last_notified_at`) · `notification` (+ `corrected_at`) · `push_subscription` · `notification_prefs` · `search_log` (ship with v1 or the weights stay guesses) · `coin_snapshots` · `story_outcome` · and the clocks `t_mint` and `t_ct`, which are designed and **persisted nowhere** — meaning today's `+31m early` badges have no prospective clock behind them.

Do not use: the `combined_views * 0.14` gain fallback or its sibling `searchSeries` delta, both invented data currently driving search ordering, and `PENDING{}`, never populated but read by five render paths. The extended search row is `.ttr-narr`; the existing `.sr-narr` is a two-column thumb-and-text row and reusing the name is a live selector collision. Roughly fifteen components here are new, not two.

Build order

1. **Migration A** — the columns above, `mint` backfilled from the DexScreener path the lookup already resolves and discards. Nothing else ships first: every screen computes its button from a number that doesn't exist yet.
2. **Rework the lookup layer** (~60 lines, gates five screens): `null` not `0` for absent numerics; `mintTime` as absolute `min(pairCreatedAt)` over all Solana pairs; stop treating `liquidity ≤ 0` as not-found; return the discriminated status. Update the three consumers.
3. **The resolver** — replace `ticker && mcap ≠ null` with `resolveAction(confirmed, tradeable)` and add the match stamp. Two files, and it kills the live "Buy \$KANG" bug on every surface shipping today.
4. **Migration B** — generated tsvectors + GIN, trigram indexes, `search_log`. No UI.
5. **/search free text**, FTS + trigram only: sections, the sentinel-escaped snippet path, the zero-result Create block, and `?q=` in the URL — the app's first linkable route.
6. **CA mode** — the four-rung ladder, the mint-in-post band, fail-closed on null `mint_time`.
7. **\$TICKER mode** — family table by mint time ascending, all six verdicts, and the availability line lifted verbatim from the create sheet so the warning can't drift between the two.
8. **⌘K palette** as a strict projection of `/search`: same parse, same query, same ranking, thirteen rows, navigate and star only. It cannot buy and cannot create; on `/search` itself ⌘K just focuses the field.
9. **Migration C + Following** — `app_user`, `follow`, `notification`, `notification_prefs`; ★ on one store with an optimistic toggle; then the Following sections and the teaching empty state. Ships before any notification — the list is useful alone and it's what the notification will point at.
10. **The notification, in-app only** — trigger, dedupe key, coalescing, live-subscribed toast, fee line, away band. The retention hook; it must not wait behind push.
11. **Web Push** — VAPID, service worker, the earned-moment prompt, quiet hours with render-time recomputation, safety-gate toggle. Measure push→buy via `?src=push` before spending more.
12. **Chips in value order** — No coin yet (blocked on `coinability_tier` being real and evaluated; do not ship a browsable, one-tap-mintable list before then) → Just made with the loud `From a story only` toggle → Trending now.
13. **The two clocks**, then `coin_snapshots`, then `story_outcome`, then Finished with All-default and the stated denominator. A record built from `lead_time_min` alone would be fabricating the product's central claim.
14. **pgvector** as a third RRF leg, A/B'd against real `search_log` click-through.
15. **Coin index (F3.2)** — CA and ticker search over mints that never touched a story, the true collision count, and the rail's live `Matches` mode.

Still open with you: whether the stories lane stays merged into Live feed or becomes its own tab — the chip rail absorbs Narratives / Tokens / New Pairs either way, but a split changes where `No coin yet` lives.

07. Wallet, onboarding, and the operator surface

What it is

Two surfaces that keep promises made elsewhere. The wallet layer makes reading Insidor free and complete and demands a wallet only at the instant an action needs one, never discarding the user's work at the gate — including the funding cliff, the highest-drop moment in the product. `/ops` is the other half: one page that says in three seconds whether the core claim is true right now, plus the honesty band that tells users on the public board when it isn't.

How it looks

Signed out, the board is the board. No blur, no overlay, no row cap, no "sign up to see more". The three tabs — Live feed, Search, You — are unchanged and every ranked table renders every column with real numbers. (Whether the stories lane stays merged into Live feed or splits out is still open with you; nothing here depends on the answer.) Three things differ: the nav's right slot renders `Connect` instead of the user chip; the comment composer becomes one quiet row on `#060607` ground with a hairline border — *"Connect a wallet to post. Reading is free — you don't need one to browse,"* cyan `Connect` inline; and the primaries relabel. It reads as a footnote, not a wall. Which is why the trade panel signed out still shows a live Jupiter quote — real `12,345.6789 HORN`, real min-received, real impact — with balance reading a dim `Connect wallet`, the note *"live route via Jupiter · connect to sign,"* and a button reading `Connect & buy 1 SOL of $HORN` in the cyan gradient rather than the lime execute gradient.

The primary is a pure function of the CONFIRMED coin count and auth never touches it: `0 → Create coin, 1 → Buy $TICKER, 2+ → See the 3 coins`. An UNSURE match renders no Buy at all — primary stays `Create the coin`, candidates collapse into a muted block: *"Possible match: \$KANG — we're not confident this is related."* The 2+ branch opens a 440px sheet on the coins-table grid — rank · ticker · mcap · liquidity · age · action — sorted CONFIRMED-first then `first_deployed` ascending, temporal order rather than mcap, because mcap is the signal that gets front-run. CONFIRMED rows get a compact lime `Buy`; UNSURE rows get an `unverified` chip and no button, absent from the DOM rather than disabled.

The connect sheet is 440px, two tiles, no footer — the options are the actions. The subtitle carries the intent in mono: *"to buy 1 SOL of \$HORN."* Tile A, `I have a Solana wallet`, shows the Phantom/Solflare/Backpack marks and `free · instant`. Tile B, `Create one for me` — *"Email or Google. We make a Solana wallet for you — you own it and can export the key any time"* — `free · ~20s`. Under Tile B only, never behind a toggle: *"New wallet, no crypto? Getting money in costs about 2.5–4.9% on a card, plus swap costs on the trade. Sending SOL from an exchange is free. We'll show both."*

The funding cliff is not a modal. It replaces the lower half of the trade panel in place, so the coin's icon, price and chart never leave the screen. Three lines head it, and the middle one is the point of the whole product:

`You need 1.02 SOL · you have 0.14 SOL` — need in ink, have in red `#FF5C6E +31m ahead of crypto twitter on $HORN · 6m ago` — the same lead badge the board used, live-counting, amber past +58m (trailing-7d p75) `0.02 SOL is held back for network fees and never spent.`

Then three tiles ordered by measured seconds-to-SOL against that counter, not by our margin. **Send SOL:** client-rendered QR, address in mono with copy, `arrives in ~30s · costs ~0.000005 SOL · no fee from us`. **Card:** the first preset is computed from the actual deficit — `ceil(deficit_sol × SOL_PRICE / 5) × 5 → $195 · covers it`, default-selected, with `$25 | $50 | $100 | $250` after it, over a four-row live recap: `Card fee -$7.60 (3.9%) · You get ~0.85 SOL · Then swap into $HORN -1.0% route · -0.5% Insidor · -0.9% impact · Ends up as ~$183 of $HORN`, closing *"Two steps, not one — there is no card-to-memecoin path on Solana."* **Swap what you have** appears only when a routable non-SOL token is held: `You have 42.10 USDC → 0.28 SOL`. The resume strip pinned at the bottom says what is actually true — *"Leave this tab open and we'll take you straight back to \$HORN"* — and a bail-out link, `Not now — watch $HORN instead`, stars the coin and its story so the drop lands as a follow rather than a bounce.

Balance lives in exactly three places: a mono lime `1.2345` in the nav chip behind a hairline divider (no `SOL` suffix, no USD); a 284px account panel the chip toggles, carrying wallet type, address with copy, `1.2345 SOL` over `$284.10`, `Add funds`, `Export wallet`, `Disconnect`; and the trade panel's balance line. Not the rail, not per row. *Every comment carries a position stamp* — a 10px pill reading `holds 2.1 SOL of $HORN` in lime, `no position` in dim grey, `sold` in amber, or `position unknown` in dim with a hairline border. It names the mint and never updates; hover gives *"position when this was posted · 14:22 UTC."*

Security is a section in `/you` , not a modal, so it is findable when nothing is wrong: wallet type, recovery method, `Insidor's access: ✓ none — we cannot move or recover your funds` , then four flat sentences — your login is your recovery, losing it without exporting means the funds are gone permanently, export runs in Privy's window and our code never receives the key, and exporting changes nothing about the wallet. Then a red pill — *"Anyone with this key owns everything in this wallet, forever"* — over a full-width `Reveal private key (3)` disabled for three seconds after it scrolls into view, which exists to stop a fat-finger reveal during a screen share.

`/ops` reuses the app shell so nobody learns a second layout under stress. Region 0 is a 54px verdict strip at exactly one table-row height: a 14px dot, `HEALTHY` / `DEGRADED` / `DEAD` in Clash Display 18px uppercase, and — taking most of the width — the sentence that names the outage: *"worst: stage_output_stall = 7 consecutive zero runs · ingest · since 14:02 UTC (2h 11m)."* The dot pulses on DEGRADED and DEAD; healthy is static. Right edge: `evaluated 18s ago · watchdog ok · healthchecks ok 41s ago` . Region 1 is the ten SLIs — `# | SLI | Value | Warn | Page | State | Breaching for | Last 60m` , mono right-aligned, state cell reusing the timing pill, row click revealing the raw SQL with a copy control:

#	SLI	PAGES AT	CLASS
1	<code>ingest_freshness</code>	30 m	dead
2	<code>stage_output_stall</code>	5 zero runs (~10 m)	dead
3	<code>gate_supply</code>	0 eligible for 10 m	dead
4	<code>board_render_fallback</code>	≥1, immediately	dead
5	<code>stage_liveness</code>	3× interval	degraded
6	<code>funnel_yield</code>	−3σ	degraded
7	<code>budget_burn</code>	1.0 before 12:00 UTC	degraded
8	<code>tick_freshness</code>	120 s	dead
9	<code>lead_time_p50</code>	< 8 m	degraded
10	<code>delta_divergence</code>	5 %	degraded

Region 2 is the funnel — ARRIVED → ADMITTED → TRACKED → TRIGGERED → CLUSTERED → PROMOTED over the last 60 minutes, conversion rates in the arrows, change pills against the 7-day same-hour baseline. A zero renders as a 22px red `0` , never dim, never an em-dash: every real outage appears here as a column of zeros, naming the broken stage without reading a log. Region 3 is the stage table — `ingest` , `ingest-tiktok` , `snapshotter` , `score` , `cluster` , `trends` , the names the code actually writes — with `Last success | Last attempt | p50 dur | Errors 1h | Rows out | Last error` , that final cell rendering `402 payment_required · twitterapi.io · 14:02:11` in red. That is the cell that was invisible for two days. Region 4 is spend per API (today, cap, burn bar, projected EOD); region 5 a 24h stacked area of `gate_reason` with a red floor rule labelled `board empty` wherever the eligible band touches zero.

The honesty band is one strip above the public board. Fresh: nothing. Lagging (15–45m), amber: *"Detection is running slow — the newest story we've seen is 22m old."* Stale (>45m), amber: *"Live detection is down. The newest story on this board is 3h 12m old. We stopped receiving posts at 14:02 UTC and we know about it."* Fallback, red: *"These stories no longer pass our freshness gate. We're showing them because the alternative is an empty screen — treat every number here as historical."* Under stale and fallback every lead badge is **removed**, not greyed; a dimmed earliness claim is still an earliness claim. Buy and Create stay enabled — Jupiter and pump.fun are independent of our ingestion and still telling the truth.

How it works

One pure gate function decides all of it: reads and follows are open; comments need a wallet but never SOL; only `buy` and `create` return `needs-sol` , and only when `balance < amount_in + 0.02` . Nothing else in the app asks about auth.

One pure `primaryAction(narrative)` returns `{kind:'create'}` / `{kind:'buy', mint}` / `{kind:'multi', n}` from `count(narrative_tickers where match_state='CONFIRMED')` and nothing else. Every CTA and every trade-panel mount reads it. The existing `topCoin()` — which picks whichever mint has the largest market cap — is deleted; a helper that silently picks a winner cannot coexist with a rule that says the ambiguous state is never rendered. A mandatory test asserts it never returns `buy` for a zero-CONFIRMED narrative.

Fees are disclosed on the path most users take. The trade recap and the confirm dialog each gain a fifth row, `Insidor fee / 0.50% (~0.005 SOL)` , in the same weight as Amount and Slippage. The Jupiter integrator fee has a 50 bps floor,

so 0.5% per trade is the cheapest number that can ever be quoted, and it is stated rather than buried. Card arithmetic is two legs and says so: provider fee read from the live quote, never hardcoded, then route + 50 bps + impact + priority on the swap. No quote, no tile — a guessed percentage on a screen about money is worse than an absent one. The create sheet does not ship its current `3% / 1%` split bars, because the system doesn't implement them; until it does, one line: *"Deployer fee share is not live yet. You will not earn fees on coins created before it ships."*

The position stamp is written server-side at insert, once: verify the token, run the address gate (any base58 32–44 char string that isn't one of this narrative's mints is blocked, no appeal — about 90% of spam for one lookup), resolve the top CONFIRMED mint, one Helius token-account read. `holds` on a positive balance, `sold` when a prior nonzero snapshot exists for the same wallet and mint, `none` when neither, `unknown` on any RPC error or timeout. Rendering `no position` on a failed read is an unverified factual claim about a person — the same class of bug as a missing timestamp rendering `age = 0` on a terminal that sells earliness. `/api/balance` requires a Privy token and rejects any owner not linked to that DID, with a 10s server cache so the 30s nav poll and the 6s funding poll collapse onto it.

Healthy means all ten SLIs non-breaching, and every SLI is an **output** measure — a heartbeat proves a process ran, not that it produced anything, and `last_heartbeat` ticked happily through all 48 hours of the outage. SLI-2 only catches a silent death after two changes: `logCycle` moves into a `finally` in every worker, so a thrown cycle still writes a row with zero output and the error; and the query left-joins a fixed list of expected worker names so a worker that dies before it can log anything emits `zero_run_streak = 999` on absence rather than emitting no row at all. Absence and zero must both breach — today only zero does, and zero requires the worker to still be alive. `worker_cycle_log` gains one `rows_out` column each stage sets from its own output metric, so the query stops assuming `ingested` means output everywhere. It doesn't, and two of the four workers that log at all would otherwise page forever from day one.

Three independent observers, because any one of them can be the thing that died: an Inngest watchdog on a 2-minute cron; healthchecks.io as an external dead-man expecting that ping, which nothing inside the system can suppress; and `pg_cron` + `pg_net` calling `evaluate_health()` in the database, which survives total loss of the app platform. The watchdog asserts its own dependencies at startup against `information_schema.columns` — an SLI querying a nonexistent column evaluates to null and paints the board green, which is exactly how the original outage stayed invisible. Dead-class breaches (1, 2, 3, 4, 8) page with `@channel` and repeat every 15 minutes until acked; degraded-class post one threaded auto-resolving message and never touch a phone. Any `payment_required` or `unauthorized` stage row pages immediately regardless of thresholds — a billing failure is not a statistical condition to be averaged.

The honesty band is one pure `tier()` reading `minutesStale` from `max(narrative_posts.first_seen_at)` — the same column and query SLI-1 uses, so `/ops` and the board can never disagree about whether the pipeline is alive. `posted_at` stays for the Age column only, where author time is the right semantic. Unknown fails to stale; **negative fails to stale too**, because `posted_at` has a known double-conversion bug producing future dates, and one bad row would otherwise render every `+31m early` badge on a board dead for two days.

Replayed against the real incident: SLI-2 warns at T+4, pages at T+10, SLI-1 pages at T+30, and the first user to hit a fallback board pages immediately. Ten minutes instead of two days.

States

STATE	WHAT THE USER SEES
Signed out	Everything readable is readable. <code>Connect</code> in nav, one footnote row on the composer, CTAs relabelled <code>Connect & buy...</code>
Privy not ready / SDK failed	Button disabled reading <code>Loading...</code> ; on hard failure, <i>"Wallet unavailable — retry or use an external wallet."</i> Board untouched either way
Unverified (UNSURE) match	No Buy anywhere. Primary stays <code>Create the coin</code> ; candidates collapsed and labelled unverified
Direct coin route, unverified story link	Buy stays enabled — the user named the mint. Story attribution reads <code>unverified link</code> ; the lead badge is suppressed
Story with zero coins	Primary <code>Create coin</code> ; <code>No coin deployed yet</code> ; no position badges, since there is nothing to hold
Brand-new token, no market data	Price, 5m, 1h, 24h, vol, liq, mcap all render a literal <code>—</code> . Never <code>0</code> , never an inferred <code>NEW</code> . Buy stays enabled; Jupiter can still route
Negative lead time	<code>-14m</code> in red with a real minus sign. Never parenthesised, never absolute-valued, never hidden; same weight as a positive badge
Coinability tier 2 (surface, no Create)	Story renders in full; the 0-coin primary degrades to <code>Watch</code> : <i>"We don't support launching a coin from this story."</i> Tier 1 never appears
Story goes never-surface after its coin launched	The <code>/you</code> position row stays and stays claimable — the money is real. The story link becomes <i>"This story is no longer shown on Insidor."</i>
Not connected, You tab	The real shell, not a redirect: <code>Ready to claim \$0.00</code> , <code>Holdings —</code> , Following genuinely populated from local storage, plus <i>"Connect to see money, positions and claims. Your saved stories are already here."</i>
Connected, first run, nothing held	Same shell, zero rows, one line: <i>"No positions yet. Your first buy shows up here with the lead time you got."</i>
Connected, zero SOL	Nav balance <code>0.0000</code> in dim, not lime; <code>Add funds</code> takes the cyan fill; any Buy or Create hands to the funding cliff
Balance fetch failed	<code>—</code> in dim. Never <code>0</code> , never a silent fallback to a cached number
Card quote pending / failed	Tile pulses with no numbers, then disables: <code>Card funding unavailable right now</code> . Send and Swap unaffected
Below provider minimum	Presets under ~\$20 hard-disabled with <code>card minimum is \$20</code> ; a covers-it amount below the minimum snaps up and says so
Waiting for deposit	Tiles collapse to one line each, lime dot pulsing. On tab blur: <i>"Waiting — we'll pick this up when you return."</i>
Wrong asset arrives	The watcher reads token accounts too: <i>"42.10 USDC arrived, not SOL. Swap it to SOL →"</i> routing into Tile 3. Silence is never the answer to an arrival
Partial arrival	Deficit updates live; the button offers <code>Buy 0.62 SOL of \$HORN instead</code> — what they can afford, minus reserve
Card settled, swap leg failed	<i>"Your \$195 arrived as 0.85 SOL and is safe in your wallet. The \$HORN route failed — retry, or hold the SOL."</i> The paid provider fee is named
Intent expired (30 min)	<i>"Your \$HORN order expired — the price moved. Re-quote?"</i>
Token untradeable while waiting	<i>"\$HORN has no route right now. Your SOL is safe in your wallet."</i> Intent dropped, not retried
Disconnected mid-session	Chip reverts to <code>Connect</code> ; any open trade panel disables execute with <code>Wallet disconnected</code>
External wallet	No export block; one row: <i>"Your keys are in Phantom. Insidor never had them."</i>
Balance > 0, never exported	Amber nudge atop <code>/you</code> : <i>"You're holding \$284 in a wallet you haven't backed up."</i> Per-session dismissible, weekly return, never modal
Watchlist merge failed on connect	Local key is not cleared: <i>"Your saved stories are still on this device — we couldn't sync them. Retry."</i> On success: <code>4 saved stories added · 11 already on your account</code>
Pipeline lagging / stale / fallback	The strip tiers above; lead badges removed entirely under stale and fallback. The Age column keeps counting, because that stays true
Recovering	Lime strip for 90s: <code>Back live — 41 new stories arrived</code> . Badges return only for stories promoted after recovery; an intent carried in from the stale window keeps its badge-less story
One platform down	<i>"X detection is down; TikTok stories are still arriving,"</i> with the platform badge inline
<code>/ops</code> evaluator stale (>3 min)	<code>UNKNOWN · the evaluator itself has not reported since 14:41</code> . A green board from a dead evaluator is never rendered
<code>/ops</code> cold start, zero samples	<code>UNKNOWN · the evaluator has never reported</code> . Every row reads <code>never reported</code> in red — empty must not look like healthy
<code>/ops</code> stage listed but never seen	<code>never reported</code> in red across the row

STATE	WHAT THE USER SEES
<code>/ops</code> no spend rows today	<code>no usage recorded today</code> in red, not <code>\$0.00</code> — zero spend and no data are otherwise identical
<code>/ops</code> schema assertion failed	<code>UNKNOWN</code> , naming the object: <code>ops_funnel.promoted absent — SLI-6 not evaluated</code>
<code>/ops</code> unauthorised	Server 404. No login prompt, no hint the route exists
Earliness ledger, n < 30	Median and percentiles render <code>—</code> with <code>not enough data (n=12)</code> rather than a noisy number
Earliness ledger, mint-clock gap	Amber strip naming the span; those stories are unmeasurable, never wins

What it needs

Used as-is: `narratives ( display_eligible , gate_reason , updated_at , combined_views ), narrative_posts ( first_seen_at` as the one ingest clock, `posted_at` for Age only, `platform_post_id` as permalink proof), `narrative_tickers ( ticker , mint_ca , mcap , liquidity , first_deployed ), worker_cycle_log , worker_usage , worker_budget_state .`

New on the wallet side: `app_user(privy_did unique, wallet_address unique, wallet_type)` — nothing maps a Privy DID to anything today; `follow(user_id, target_type, target_id)` — the watchlist is two in-memory Sets and dies on refresh; `holding(user_id, mint, narrative_id, tokens, cost_sol, entry_price, opened_at, minutes_early)`, where `minutes_early` is signed and renders negative; and `comment(..., snap_views, snap_coin_count, snap_age_min, snap_top_mcap, snap_position_lamports, snap_position_mint, snap_position_state)` — the stamp is unrecoverable retroactively, so it ships before comments do. Whether `trade_intent` also lives server-side is open: it survives a device switch and gives a real drop-off funnel, at the cost of storing unexecuted trading intentions.

New on the ops side: `narrative_tickers.match_state ('CONFIRMED'|'UNSURE'|'REJECTED')` plus `match_score`, `mint_time`, `evidence`, and `source ('cashtag'|'llm'|'mint_in_post')` — the primary button is uncomputable without `match_state`, and missing provenance is the root cause of the \$KANG class; `narratives.promoted_at` and `earliness_eligible`; `coin_mints` and `ct_mentions`, the two clocks; `ops_stage_run`, `ops_event`, `ops_sli_sample`, `ops_funnel`, `ops_budget_cap`; and one `worker_cycle_log.rows_out` column.

Three flags to keep. `worker_pipeline_state`, `worker_crashes` and `worker_usage.cost_breakdown` are defined in SQL but absent from the 2026-07-23 live schema probe, so SLI-5, the errors rail and the spend region may be querying objects that were never migrated — verify the applied state, not the SQL files. `worker/trends.js` currently inserts `refreshed / open / trend_enriched`, which are not columns on `worker_cycle_log`; that insert fails into a swallowed warning, so `trends` has no cycle history at all right now. And `funnel-log.js` is a generic stepper with two call sites and arbitrary labels — a printing convention, not the six-stage taxonomy. Nothing anywhere emits `promoted`; there is no PROMOTE event in the repository. The funnel must be *defined* before it can be persisted.

Build order

- The incident fix — two hours, no new infra.** Widen the retry predicate to `{408, 425, 429, 500, 502, 503, 504}` so a 402 is loud instead of fatally silent; add `stage-run.js` + `ops_stage_run` and wrap all six real stages; move `logCycle` into a `finally`; add `rows_out`; fix the `trends` insert; emit `ops_event(kind='board_fallback')` from the `showingIneligible` the code already computes and discards. Add the one-line `.dim{color:var(--dim)}` rule while you're there — three honesty states render as confident primary text without it.
- match_state as a schema-only change**, defaulting every existing row to `UNSURE`, plus `primaryAction()` with its test and the deletion of `topCoin()`. This gates the whole wallet track: until it lands, every Buy button in the product is a market-cap guess.
- Watchdog and dead-man in one PR** — samples, events, budget caps, the 2-minute cron over the seven SLIs needing no new clocks, the Slack dead/degraded split, the healthchecks ping, the startup schema assertion. Alerting without a dashboard is useful; a dashboard nobody watches is not.
- Define, then persist, the funnel.** Name the six boundaries against real code points, make PROMOTE an explicit event stamping `promoted_at` once behind a `coalesce(promoted_at, now())` guard, then add the table and the in-database third observer.
- /ops regions 0–5** behind a server-side DID allowlist.
- The honesty band** — `tier()` with tests covering null and negative, the strip container, and the load-bearing part, lead-badge suppression under stale and fallback. Delete the interval that writes the literal string `now` every four seconds in the same PR.

- 7. **Wallet provider and gate** — Privy pinned to Solana-only, the 50-minute token refresh feeding realtime auth (without it every user silently drops off the live feed after an hour, the most likely launch-day bug), `gate.ts` with tests, `app_user` + `follow`, the footnote gate component, and a local follow set whose merge clears local storage only on a 2xx.
- 8. **Connect sheet, both paths**, plus the nav balance, the ported account panel, an authed and cached `/api/balance`, real signed-out quotes, and the intent handoff. The `Insidor fee` row lands here, on the ordinary funded buy.
- 9. **The funding cliff** as its own release, instrumented per tile: deficit-aware presets, the earliness counter, the two-leg cost line from live quotes, the watcher and auto-resume.
- 10. **Export and security** in `/you`, with the dwell timer and the never-exported nudge.
- 11. **The clocks** — `coin_mints` from a supervised PumpPortal stream with gap detection, `ct_mentions`, `earliness_eligible`. Nothing about lead time is measurable before this.
- 12. `/ops/earliness` and the **public record** off one identical query, with `All` as the default filter asserted in CI.
- 13. **The comment position stamp**, shipping with moderation in the same release and never before it.

4. Shared components

Five things recur across every surface. Each has exactly one implementation, in `shared/` or `features/*/model/`, imported everywhere. Where a component renders differently in two places, the product contradicts itself — and on this product every contradiction is about money or about a claim.

4.1 The primary-action resolver

`features/coins/model/action.ts` — pure, no I/O, mandatory tests. **The single implementation of the one rule.**

Every CTA on every surface calls it and nothing else: the posts row, the coins row, the story page, the sticky bar, the mobile action bar, the search row, the BEST MATCH strip, the Following row, the mint toast, the create entry points.

```
type Action =
  | { kind: 'create', href: string }
  | { kind: 'buy', mint: string, ticker: string }
  | { kind: 'multi', n: number }
  | { kind: 'view', mint: string, reason: 'unsafe' | 'ambiguous_symbol' }
  | { kind: 'none', reason: 'no_create' };

function primaryAction(story, matches, safety): Action
```

Two inputs, not one. A match enters the confirmed set only when `verdict IN ('CONFIRMED','ADOPTED')` **and** `mint_time ≠ null and ticker ≠ null and` it passes the hard safety gates. `UNSURE` never enters the count in any branch. A confirmed-but-unsafe coin returns `view`, rendered as an outline button plus a red chip naming the failing gate.

Three helpers are **deleted**, not deprecated: the `ticker && mcap ≠ null` tradeability test (which flips a four-minute-old confirmed coin back to Create), the mcap-max coin picker (precision by market cap is the failure D1 exists to prevent), and any local re-derivation of the count. A helper that silently picks a winner must not survive alongside a rule saying the ambiguous state is never rendered.

Labels are produced by the resolver, not by call sites: `Create coin` · `Buy $HORN` · `See the 3 coins` · `See the coin` · no button.

4.2 The coin-confidence stamp

`.match-stamp` — mono 9px uppercase, .06em, 1px border, radius 4, 1px 5px padding. Four states, one component, identical on the coins lane, the search results, the CA card, Trending, the Following sections, the story evidence popover and the coin page origin band.

STATE	COLOUR	WORDS
CONFIRMED	lime on <code>rgba(155,240,60,.08)</code>	<code>matched · image + name</code>
UNVERIFIED	amber on <code>rgba(255,184,77,.08)</code>	<code>we could not confirm this</code>
ADOPTED	cyan	<code>existing coin · adopted by this story</code> + the coin's real age , always
NO STORY	<code>--dim</code> on <code>--line</code>	<code>no story</code> — no border, no box, the quietest thing in the row

Rules that hold everywhere:

- **The numeric score is stored and never sent to the client.** Not in a tooltip, not in a data attribute, not in the API response. A user shown "73%" next to a Buy button does their own thresholding and the abstain band stops functioning.
- **Confidence is categorical prose.** Channels are named in words — `ticker matches · name does not · image not checked · minted 3d after the post` — and channels that did not run read `not checked` , never `failed` . The denominator is the channels that actually ran: *"1 of 2 channels strong"* , not 1 of 3.
- **Clicking an UNVERIFIED stamp opens the evidence popover, not the destination.** Navigating on an unestablished claim asserts it. A second, deliberate click proceeds, and it proceeds to the coin page, where the two-step confirm lives.
- **The stamp is a property of the (story, mint) pair**, not of the mint. A coin confirmed for story A and unsure for story B does not inherit a red banner from B; a warning renders only for the pair the user arrived through, and with no referrer the coin resolves against its highest-confidence confirmed story with a neutral secondary line.
- **UNVERIFIED never means hidden and never means pushed.** It suppresses Buy, it is shown at full opacity with a placeholder icon rather than the token's own logo, and it never demotes a Create button or applies pressure toward a purchase.

4.3 The market-data pending state

Three visually distinct renders, because collapsing them is how a terminal misleads. One formatter, `shared/format/pending.ts`, applied to every numeric cell in the product.

RENDER	MEANS	EXAMPLE
<code>-</code> in <code>--dim</code>	We cannot obtain this.	dev%, sniper%, bundle% — permanently unmodelled
<code>-</code> in <code>--dim</code> with <code>title="coin is 41m old"</code>	The window has not elapsed.	24H on a 41-minute coin
<code>..</code> in <code>--dim</code>	Expected imminently, waiting.	price before the first trade

`$0`, `0.0%` and a green tick are never rendered for absent data, anywhere, at any age. `$0` claims worthlessness; `-` claims ignorance. A `0.0%` change pill reads as "flat", which is a different assertion from "undefined" and is the single most misleading render available on a coins board. The underlying lookup returns `null` for absent numerics rather than coercing to zero, and every consumer is updated in the same PR.

Three consequences that are not obvious: - **A missing timestamp never renders as new.** Unknown age renders `-`, forces the confidence stamp to NO STORY, and suppresses Buy. `return 0` on a missing pair-creation time renders a token of unknown age as "brand new" on a terminal that sells earliness, which is the most dangerous single line in the codebase. - **Unknown never renders as safe.** Absent authority fields are `null`, not `revoked`. Three-valued logic throughout: absent → unknown → `-`; present-and-empty → revoked; present-with-a-pubkey → active. And a check every token passes by construction gets a dim non-signal line, not a green tick — a tick everyone earns teaches users that green means safe, which inverts the check’s meaning. - **A failed read never renders as a fact about a person.** The comment position badge renders `position unknown` with a dashed border, never `no position`, which would be a fabricated credential in the author's favour.

Loading is distinct from all three: `.sk` skeleton bars at the cell's real width, so the layout never jumps.

4.4 The lead-time claim

`shared/lead/` — one formatter, one component, one suppression rule. It appears on the posts board AGE cell, the coins lane STORY cell, the story page verdict bar and proof timeline, the coin page origin band and position card, the create sheet's locked story, the search story row, the Finished record, the mint notification, the success screen and the share card. **If it renders differently in any two of those, the product is lying in one of them.**

```
t_crypto = min(t_mint, t_ct)
lead_min = (t_crypto - promoted_at) / 60000
```

CONDITION	RENDER	WEIGHT
<code>lead > +2m</code>	<code>+31m early</code>	cyan, <code>.lead-badge</code>
<code>-2m ≤ lead ≤ +2m</code>	<code>same minute</code>	dim
<code>lead < -2m</code>	<code>-12m late</code> (U+2212, tabular)	red, <code>.lead-badge</code> — identical geometry and identical size
<code>t_crypto</code> null, both clocks verified live	<code>+2h14m and counting</code>	cyan, live
either clock in a gap window	<code>unmeasurable</code>	amber, dashed
<code>promoted_at</code> null	<code>not promoted</code>	dim
<code>earliness_eligible = false</code>	<code>already hot</code>	dim, dashed

Seven rules, all load-bearing:

- Negative renders at the same visual weight as positive.** Not a quieter component, not a smaller size, not grey where positive is coloured. A third of real detections are late; a record with no left tail is marketing. The sign is always shown, the minus is a real minus, and the value is never absolute-valued or parenthesised.
- No hedging vocabulary.** Not "narrowly behind", not "slightly late". `-12m` in red, and the sentence is *"Crypto got here first."*
- `promoted_at` is set once at PROMOTE and is NULL for every pre-existing row.** No backfill from ingest or cluster timestamps — those differ from promote by roughly the amount of lead the pipeline produces, so backfilling from them fabricates the number the product is sold on. NULL propagates to `earliness_eligible = false` and to `not promoted`, forever.
- Both clocks must be verifiably alive for a positive open-ended claim.** `+2h14m and counting`, live-incrementing in cyan, generated by a dead poller is the worst output this system can produce. Absent evidence of liveness the cell reads `— / CLOCK DOWN` naming the blind sensor. Gap detection is symmetric: the mint stream and the crypto-account poller each have a gap table, and either one overlapping the measurement window forces `unmeasurable`.
- The claim is suppressed entirely when the pipeline is stale.** `showLeadClaims` (§3.1) removes every badge on every surface under `stale` and `fallback` — removed, not dimmed. A dimmed earliness badge on a dead pipeline is still an earliness claim.
- Our lead and the user's lead are different numbers and never share a component.** `t_crypto - promoted_at` is a historical fact about the detector. `t_crypto - first_buy_at` is the user's, frozen at fill. And `now - promoted_at` against the coin state is the *live* question — *is the head start still there* — which is what a trader standing on a page actually needs and which renders as its own cell: `promoted 6m ago · no coin yet · you are ahead / crypto caught up 7m ago`. The sticky action bar may show ticker and price. It may never show the frozen `+31m`.
- The proof travels with the claim.** Wherever the number is stated at any size above a chip, two external permalinks are one click away — the original post and the first crypto post — plus the provenance line: `measured against 41,208 mints streamed + 300 crypto accounts polled every 2 min · t_crypto = first of either`. The claim is verified by two public timestamps belonging to somebody else, not by trusting us. On the share card and the public record the same rule applies: the description is templated off the real state, and a negative lead does not emit a boast.

4.5 The fee line, and the staleness band

Two smaller shared pieces that must also be identical everywhere.

The fee line. `Insidor + Jupiter fee / 0.60% · $0.28` — a row in the recap block at **exactly the typographic weight of Price impact and Min received**. No colour, no box, no badge, no acknowledgement checkbox. A 10px ⓘ opens the split, including the constraint stated plainly: *"0.50% is the minimum Jupiter permits on integrator fees — it is not a number we chose."* It renders in the trade panel, **again as a full uncollapsed row in the confirm dialog before any signature**, inside a toast-mounted trade panel, in the funding sheet's two-leg arithmetic, and — as Insidor's share of the creator fee stream — in the create flow's cost popover, in both denominations. Two exposures, ambient and pre-signature, neither shouted. When the SOL price is unavailable it falls back to the SOL amount, never to `$0.00`.

The staleness band. One `tier()` function, one container style, four tiers, five feeds. Any surface that renders live numbers renders the band for the feed it depends on, with its own age counter, and degrades only what that feed actually affects. The board's price feed going stale does not disable navigation; our ingestion going stale does not disable trading, because Jupiter and pump.fun are independent of us and are still telling the truth. Nothing anywhere fabricates freshness — no spinner implying work, no skeleton standing in for stale data, no interval writing the literal string `now`.

5. What is not in v1

Explicitly, so nobody builds it by accident.

Named cuts. - Trenches. No separate high-risk lane. The Fresh band plus SHOW RISKY covers the intent, and a second surface with a different risk posture doubles the number of places a bad coin can be presented as normal. - **Ranking and rev-share.** No creator leaderboard, no referral tiers, no user-facing revenue split beyond the coin's own fee stream. A ranking creates the incentive to farm the thing being ranked, and the two things worth farming here are launches and comment volume, both of which we are trying to keep clean. - **Social profiles.** No user pages, no bios, no follower graphs, no avatars beyond the deterministic wallet-derived swatch. A wallet, a handle and a position badge is the whole identity model. The stamp is what carries reputation, and it is non-gameable precisely because it is not a profile. - **Flex cards.** No PnL screenshots, no "I was 41 minutes early" shareables beyond the one launch card, which exists because it is a growth loop and is branded honestly. A product publishing its misses cannot ship a share format that only exists for wins.

Cut because the critiques showed they are not ready. - Δ MINT as a Fresh-band column. It needs a first-trade price captured by a trade stream, not by a snapshot poller that can be 300 seconds late on a six-minute-old coin. Until the stream writes it immutably, the Fresh band ships volume-since-mint with the elapsed window as a superscript, which is honest at any grain. - **Peak market cap, and the flat outcome.** Both need a coin market-cap time series that does not exist. Finished ships four outcomes — early, late, no coin, unresolved — and omits the Peak column entirely rather than labelling a last-observed value "peak". - **Trending below a 24-hour age floor.** A 24h-weighted score over coins younger than 24 hours is 85% undefined. The 18-hour gap between the Fresh band's 6h ceiling and Trending's floor is covered by nothing in v1, deliberately. - **Comment upvotes and threaded replies.** Votes are farmable and the stamp gives non-gameable ordering for free. Ship the stamp for eight weeks before deciding whether a vote button earns its place; replies stay flat. - **Perceptual image matching as a confirming channel.** It ships as evidence in the popover (the media pair renders because it is the proof surface regardless) but does not contribute to a verdict, and its absence is silent — never claimed as a failed check. - **Wallet-behavioural risk signals** — dev holding, sniper, bundle, insider percentages. They need first-slot transaction analysis. They render as `—`, are listed by name in the risk popover as `not available at launch`, and count against the chip's obtainability denominator so a clean chip cannot be manufactured by absence. - **The poster fee share.** The sharing config is created on every launch so the option is preserved permanently; the shares are not written. The create sheet's current 3%/1% markup is deleted, because it asserts a split the system does not implement. - **Direct-to-mint fiat funding.** SOL-only. Whether the onramp can deliver an arbitrary SPL mint is unverified, and shipping on the assumption makes the normie path fiction. - **Reddit, Farcaster and Telegram.** Two parsers exist. Three dead filter chips reading zero forever is the same category of lie as a fabricated thumbnail. - **Cross-device saved-search and cross-device funding intents.** Both need server-side intent storage; localStorage covers the single-device case, and the copy states what is actually true rather than promising a hand-off that does not happen. - **A separate /story/[id]/coins compare route.** The N-coins sheet on the story page is the destination. One surface, one sort rule. - **Push on iOS.** Web push requires the PWA to be installed to the home screen.

v1 is Android and desktop for push; iOS users get in-app plus the while-you-were-away band, which already delivers the journey end to end.

Two things that are not cuts but are gates. No public board of any kind ships before coinability is populated and evaluated — the "No coin yet" list is a browsable, one-tap-mintable index and is the most exposed surface in the product. And no earliness number is rendered anywhere before both clocks are running with gap detection, because a lead time computed from a single live sensor is not a measurement.

6. Open questions for the founder

Ten genuine product judgement calls. Everything else in this document has been decided.

1. The poster fee share. Does 40% of a coin's fee stream go to the person whose post it came from, at an address derived from their handle, whether or not they have ever heard of Insidor? It is the most defensible thing about the product and the largest legal surface, and it is irreversible per coin: a mint launched without a share configuration can never have one added. The engineering answer is settled — the config is created on every launch so the option stays open at ~0.004 SOL each — but the shares cannot be written until you decide, and every coin launched before you do is permanently split-less. Deciding "no" later costs nothing; deciding "yes" later is impossible.

2. The HIT threshold. A story's outcome is HIT at peak market cap $\geq X$ and DUD below it. That number decides what fraction of the public record reads as a win. Set it once, from the existing corpus rather than from taste, and never move it — a threshold that moves is a track record that can be edited.

3. The landing lane. Posts or Stories? "Front door" and "where earliness is felt" both argue Posts. But a normie arriving from a TikTok link is better served by a story with a coin already attached, and a 40-row pre-coin board where 90% of buttons say [Create coin](#) is the wrong first impression for two of the three named users. This may want to be a per-referrer default rather than a product-wide one.

4. The dev buy in the create form. Currently kept, defaulted to None, relabelled [Buy some yourself \(optional\)](#), and stated plainly to execute at the same price in the same transaction. The argument for cutting it entirely is that it is the one screen that makes a beginner correctly ask whether they are running a scam. Keep the honest middle, or cut it?

5. Storyless creation. The create form is reachable only from a story, which is what keeps creation coherent — and which also means a user cannot make a coin about something we missed. v1 blocks it and offers [Watch this search](#) instead. That forecloses a real intent, on the surface where a user is most motivated. Accept the loss, or allow a synthesised single-post narrative and take user-authored rows into the pipeline's own table?

6. Displaying the take rate. Jupiter's platform fee is published at 50 bps and measured live at 10. If they flip it back, our disclosed total silently doubles from 0.60% to 1.00%. Display the live response value (correct, but the number moves under users) or a conservative "up to 1.0%" (stable, but overstated most of the time)? There is a trust cost either way.

7. Shipping the From-a-Story band thin. At launch, with recall targeted at 0.55–0.70 and a 15–30% abstain band by design, the band that best demonstrates the thesis will be the emptiest — plausibly three to eight rows. Ship it thin, which is honest and consistent with the record we publish, or hold it behind a flag until the matcher has volume, which is more flattering? This should be decided deliberately, not by default.

8. The coinability boundary for a person who went viral once. The rule permits public figures and stage names and forbids identifiable private individuals. The modal Insidor story is someone who was private until Tuesday. This needs a written rule from you, not a model judgement, because it is the tier that will actually bite and it is the one where being wrong is unrecoverable.

9. A launched coin whose story is later reclassified. The coin exists on Solana, the creator's wallet is accruing fees, and the story has vanished from every board under the never-surface tier. Does the position row still render? Does the fee claim still work? What does the user see where the story link was? There is no defensible default here and creation should not ship without the answer.

10. False-block tolerance on the honeypot check. The reverse-quote sell-route check will occasionally block a legitimate buy where the route is momentarily unavailable rather than genuinely absent. What false-block rate is acceptable? It is a commercial decision about how much revenue you are willing to forgo to never let someone into a honeypot, and it wants a number rather than an instinct, because the threshold will otherwise drift toward whichever side complains first.

7. Risk register

RISK	SEVERITY	STANDING
X ingestion is dead in production since 25 Jul 08:54 — no live data reaches the site	Blocking	Check twitterapi.io balance and Vercel <code>X_API_KEY</code> . Nothing in Weeks 1–3 can be validated until this is restored. See §2.
No mint address stored for any coin — token-lookup resolves one and throws it away	Blocking F2	One-column migration + backfill. A swap cannot be built without it.
Ticker-only coin matching yields false positives — unrelated stories linked to a \$1.8B token	High	Present every match as a candidate until the F3.2 indexer lands. A wrong "Buy" button on a memecoin terminal is a user losing money.
X ingestion is the entire cost story — \$0.005/post read, ~\$200 beta → up to \$10,000 at 50k DAU. Everything else together is under \$1,200/mo at 50k.	High	The 2M read/month ceiling is both the budget <i>and</i> the product limit. Curate accounts; don't chase the firehose.
X Display Requirements forbid attaching third-party actions to a displayed Post	High	Mitigated by A10 — attach to the narrative. Keep it that way; it's a hard rule.
X revoked API access for "InfoFi" apps (15 Jan 2026) — apps attaching financial rewards to posting	High	Our distinction is that payout triggers on <i>trading volume</i> , never on posting — structurally a royalty, not a bounty on amplification. That distinction is ours, not X's, and untested. Counsel must read the mechanic against the actual Developer Agreement.
TikTok ingestion is third-party scraping (Apify), already live in code	High	Undocumented decision, now documented. No lawful first-party discovery API exists for TikTok. Counsel should see this.
Paying a poster who never consented	High	Now technically easy, which makes the legal question binding. Right of publicity is state law, no federal statute, consent is the primary defence. Ship the opt-out registry and the "not created or endorsed by" disclaimer in the same release as the split.
Rev-share for commentary	High	FTC Endorsement Guides (\$51,744–\$53,088/violation) + CFTC Rule 180.1 exposure when we also earn fees on the promoted thing. Build the score, display the score, do not pay for the score. Correctly LATER.
<code>sharing_config</code> permanently disables <code>collect_creator_fee_v2</code> for that mint	High	If the distribution cranker breaks there is no fallback collection path . Build and monitor it before shipping a single shared coin.
PDA seed mismatch: <code>[b"creator-vault", creator]</code> on Pump vs <code>[b"creator_vault", coin_creator]</code> on Pump AMM	Medium	Hyphen vs underscore. Easy and expensive.
<code>/api</code> open proxy to paid keys	Medium	Fix in Week 0. Currently anyone with the URL can drain Birdeye CU and Jupiter quota.
Zero tests on the scoring/clustering heuristics	Medium	Can't tune <code>meme-score</code> or <code>cluster-engine</code> with any confidence about what broke. First tests in Week 0.
APIs in this space move weekly	Medium	The build spec's adversarial pass refuted 15 of 26 specific claims — not the conclusions, but exact endpoints, prices and method names. Treat every code identifier in this document as needing confirmation on the day it's used.